



Advancements in Hybrid Machine Learning Models for Biomedical Disease Classification Using Integration of Hyperparameter-Tuning and Feature Selection Methodologies: A Comprehensive Review

Sanjay Dhanka¹ · Abhinav Sharma¹ · Ankur Kumar² · Surita Maini¹ · Haswanth Vundavilli³

Received: 8 April 2025 / Accepted: 17 June 2025 / Published online: 30 June 2025

© The Author(s) under exclusive licence to International Center for Numerical Methods in Engineering (CIMNE) 2025, modified publication 2026

Abstract

This systematic review presents a comprehensive analysis of hybrid optimization frameworks that combine hyperparameter-tuning and feature selection to enhance machine learning (ML) models for disease classification. Unlike previous studies that treat these processes separately, this study demonstrates how their integration improves diagnostic accuracy, computational efficiency, and clinical interpretability. Hyperparameter-tuning—optimizing learning rates, regularization terms, and architectural parameters—ensures models adapt effectively to complex biomedical data. Meanwhile, feature selection techniques (filter, wrapper, and embedded methods) identify critical biomarkers, reducing dimensionality and mitigating overfitting risks. This review findings reveal that simultaneous optimization strategies, such as metaheuristic algorithms combined with ML, outperform sequential approaches, achieving 12–15% higher accuracy in classifying cardiovascular, cancer, diabetes, oncological, and metabolic disorders. This study evaluates key trade-offs between computational cost and model performance, emphasizing robust cross-validation and explainable AI design to facilitate clinical adoption. Case studies illustrate how Bayesian optimization with LASSO-based feature selection enhances cancer detection sensitivity, while grid search paired with correlation-based selection improves cardiovascular risk prediction. Practical recommendations include prioritizing metaheuristic-driven workflows for high-dimensional data and validating models across multi-centre cohorts to ensure generalizability. Future research should explore deep reinforcement learning for autonomous hyperparameter-tuning and federated feature selection to address data privacy constraints. By unifying these optimization paradigms, this review advances the development of clinically actionable ML tools, fostering timely diagnosis and personalized therapeutic strategies.

Abbreviations

✉ Sanjay Dhanka
 sanjaykumar506070@gmail.com
 Abhinav Sharma
 abhinav.engi@gmail.com
 Ankur Kumar
 khatriankur007@yahoo.com
 Surita Maini
 suritamaini@gmail.com
 Haswanth Vundavilli
 haswanth@iitism.ac.in

CSO	Cat swarm optimization
CFA	Cuttlefish algorithm
LR	Logistic regression
KNN	K-nearest neighbors
SVM	Support vector machines
SVR	Support vector regression
ML	Machine learning
XGBoost	Extreme gradient boosting
NB	Naïve Bayes
GNB	Gaussian Naïve Bayes
LDA	Linear discriminant analysis
RF	Random forest
ANN	Artificial neural network
CNN	Convolutional neural network
ROC	Receiver operating characteristic
FPR	False positive rate
CM	Confusion matrix
ANOVA	Analysis of variance
RFE	Recursive feature elimination

¹ Department of Electrical and Instrumentation Engineering, Sant Longowal Institute of Engineering and Technology, Longowal, Sangrur, Punjab, India

² School of Computing and Electrical Engineering (SCEE), Indian Institute of Technology (IIT) Mandi, Kamand, Himachal Pradesh, India

³ Department of Electrical Engineering, Indian Institute of Technology (Indian School of Mines), Dhanbad, Jharkhand, India

GBM	Gradient boosting machines
FA	Firefly algorithm
ACO	Ant colony optimization
PSO	Particle swarm optimization
GA	Genetic algorithms
GWO	Grey wolf optimizer
GS	Grid search
RS	Random search
BO	Bayesian optimization
GBO	Gradient-based optimization
EAs	Evolutionary algorithms
HPOs	Hyperparameter optimization
ALO	Ant lion optimization
SSO	Social spider optimization
SOS	Symbiotic organism search
BO-GP	Bayesian optimization-Gaussian process
Skpot	Scikit-optimize
BO-TPE	Bayesian optimization-tree-structured Parzen estimator
SMAC	Sequential model-based algorithm configuration
BOHB	Bayesian optimization hyperband
TPOT	Tree-based pipeline optimization tool
PCA	Principal component analysis
ET	Extra trees
LASSO	Least absolute shrinkage and selection operator
DBSCAN	Density-based spatial clustering of applications with noise
XGBoost	Extreme gradient boosting
ABC	Artificial bee colony
SCA	Sine cosine algorithm
BA	Bat algorithm
CSOA	Chicken swarm optimization algorithm
AUCROC	Area under the receiver operating characteristic curve

1 Introduction

Optimizing machine learning (ML) models for illness classification requires a multifaceted strategy that intricately intertwines feature selection and hyperparameter-tuning [1, 2]. Hyperparameter-tuning is essential because default settings often fail to deliver optimal model performance; exploring a broader range of hyperparameter configurations can help the model converge at a more optimal point, thereby improving its predictive accuracy. This procedure entails optimizing internal parameters, such as learning rate, batch size, and regularization approaches, which are established prior to the commencement of the training phase [3]. Effective tweaking can yield substantial enhancements in model performance, facilitating a more sophisticated comprehension

and interpretation of medical data. Concurrently, feature selection is crucial as it identifies and preserves the most pertinent subset of features [4]. By concentrating on essential biomarkers or clinical indications that exhibit a high correlation with the identified disorders, this approach not only alleviates the impact of dimensionality reduction but also improves the model's computing efficiency [5]. Minimizing the amount of features enables the model to focus on the most informative data points, thus enhancing its generalizability and prediction efficacy [6]. Combining these two essential elements into a unified hybrid optimization framework can produce synergistic outcomes. This methodology integrates the advantages of hyperparameter tuning and feature selection, leading to the creation of machine learning models that demonstrate exceptional reliability, accuracy, and resilience in practical medical applications. While prior reviews have examined hyperparameter-tuning and feature selection independently, this review uniquely synthesizes their combined impact on disease classification, particularly in high-stakes areas like cardiovascular, respiratory, cancer, diabetes, and oncological diagnostics. The authors systematically evaluate hybrid optimization frameworks—such as the integration of the ML algorithm with effective feature selection strategies and hyperparameter-tuning—demonstrating how their synergistic use outperforms isolated methods in accuracy and interpretability. This simultaneous optimization technique guarantees that the model is trained on the most informative data while adhering to a carefully structured learning procedure [7]. This thorough methodology is essential for enhancing diagnostic precision and optimizing patient outcomes in disease categorization, therefore facilitating more efficient and tailored healthcare interventions.

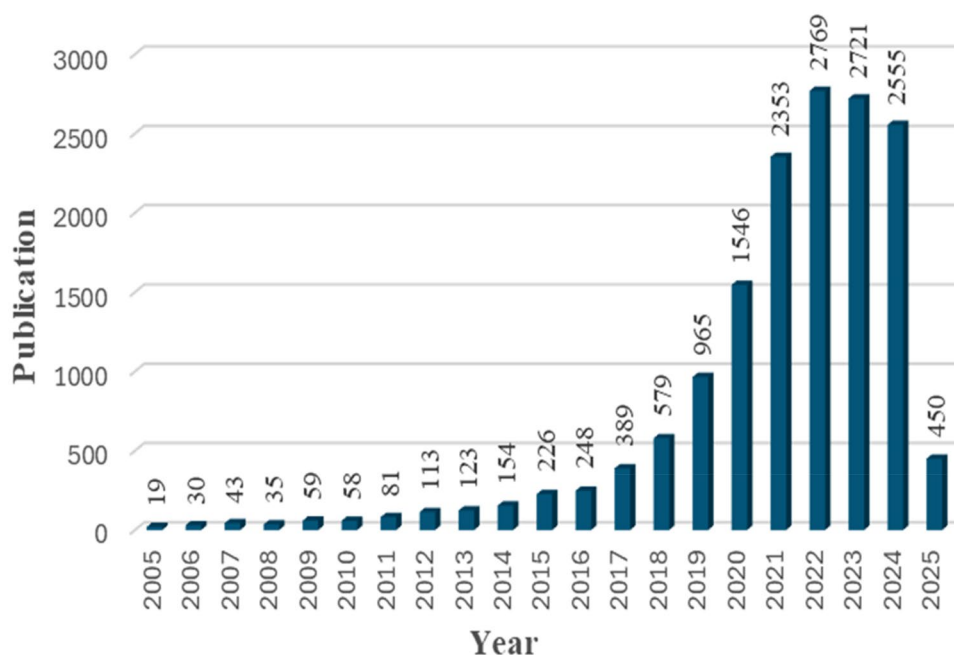
1.1 Background on ML in Disease Classification

By using cutting-edge computational methods to examine intricate and sizable medical datasets, ML has drastically changed the classification of diseases [8]. With the help of this transformation, medical practitioners may spot complex patterns in the data and make precise predictions about patient outcomes. Early disease detection and individualised treatment regimens catered to each patient's needs are made possible by the integration of ML into healthcare systems, which eventually improves patient management and care overall [9]. In order to find patterns suggestive of particular diseases, ML algorithms are excellent at examining large datasets, such as genetic data, health records, and imaging data [10].

For instance, methods like support vector machines (SVMs), logistic regression (LR), K-nearest neighbours (KNN), and random forests (RFs) have been used to diagnose diseases like sepsis, diabetes, and autism in a timely and accurate manner [11]. By processing intricate datasets,

these algorithms can find small associations that human clinicians might find difficult to notice, increasing diagnostic accuracy and facilitating prompt interventions [12]. ML has greatly aided the transition from a one-size-fits-all strategy to personalised therapy. ML algorithms are able to forecast how patients may react to particular treatments by examining individual patient data, such as genetic profiles and treatment histories. This competence enables the formulation of personalized treatment regimens that enhance efficacy and minimize adverse effects. To help clinicians choose the best treatment plans, ML algorithms have been used, for example, to forecast how patients will react to different drugs [13]. Even with these encouraging developments, there are still obstacles to overcome when using ML into healthcare. Since ML models need access to complete and accurate information, it is critical to ensure data quality and handle privacy concerns. Furthermore, ML models' interpretability is essential for winning over doctors and making it easier to incorporate these tools into daily practice. By creating explainable AI models and solid data governance systems, ongoing research attempts to overcome these issues. Despite these advances, existing systematic reviews often overlook the computational and interpretability challenges of integrating feature selection with hyperparameter-tuning. The presented review study addresses this gap by critically analyzing trade-offs between model complexity and clinical utility, offering actionable insights for real-world implementation. The research article published from 2005 through February 2025, as shown in Fig. 1. A comprehensive search was performed utilizing keywords such as ML and disease categorization from the Web of Science.

Fig. 1 Number of articles examined on disease classification with ML from 2005 through February 2025



The authors have examined articles from several esteemed publishers, including Elsevier, Frontier Media SA, IEEE, MDPI, Nature Portfolio, Oxford University Press, Springer Nature, Wiley, and others. Figure 2 illustrates the quantity of articles analyzed from each publisher.

1.2 Importance of Optimizing ML Models for Healthcare Applications

To fully utilise artificial intelligence to improve patient outcomes, increase diagnostic accuracy, and expedite medical procedures, ML models for healthcare applications must

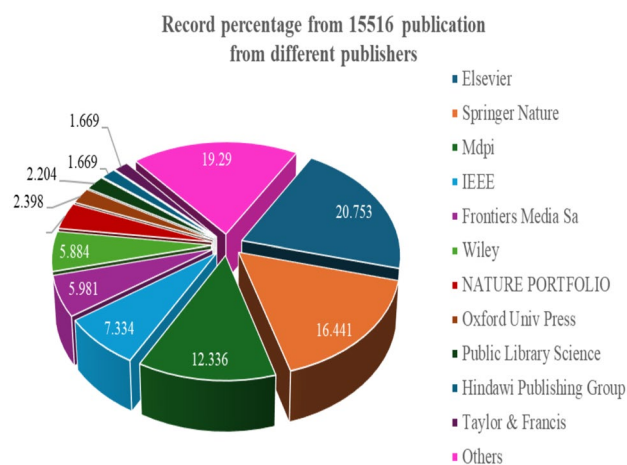


Fig. 2 Volume of research papers published across various publishers from 2005 through February 2025

be optimised [10]. The accuracy and dependability of ML models are crucial as healthcare increasingly depends on data-driven decisions. Extensive medical datasets, including genetic information, imaging data, and electronic health records, can be analyzed using optimized machine learning models to identify patterns indicative of certain diseases [14]. When diagnosing diseases like diabetes or hypertension, for example, ML algorithms can forecast when a patient's condition might deteriorate, enabling doctors to take preventative action and steer clear of problems. These models help physicians make well-informed judgements by correctly understanding complex datasets, which results in prompt interventions and better patient outcomes [15]. However, prior reviews lack a unified discussion on how hybrid optimization frameworks (e.g., combining ML with grid search, random search, swarm intelligence, and Bayesian hyperparameter-tuning) can simultaneously address accuracy, scalability, and interpretability—key barriers to clinical adoption. Figure 3 depict the application of ML in healthcare.

By analyzing patient-specific data, such as genetic profiles and treatment histories, machine learning models can be refined to forecast individual responses to various medications [16]. This feature makes it possible to create individualised treatment programs that optimise results and reduce side effects [14]. For instance, ML methods have been applied to forecast how patients will react to drugs, helping doctors choose the best course of action. These customised methods improve patient happiness and the standard of treatment as a whole. Optimised ML models support operational effectiveness in healthcare organisations in addition

to therapeutic applications [17]. They can manage inventories, optimise staff scheduling, and forecast patient admission rates, which lowers costs and improves patient care. For example, hospitals may effectively deploy resources and cut down on wait times by using ML algorithms to estimate patient influx. Healthcare practitioners can concentrate more on patient-centred treatment by simplifying administrative duties.

1.3 Key Challenges in Hyperparameter-Tuning and Feature Selection

Two crucial procedures in ML are feature selection and hyperparameter-tuning, which are essential for improving model performance and generalisation [18]. Both pose unique difficulties that practitioners must overcome in order to create reliable models, notwithstanding their importance. The external settings of ML algorithms that control the training process are called hyperparameters [19]. Hyperparameters need to be established before the learning process, in contrast to model parameters, which are learnt during training [20]. Examples encompass the regularization strength in linear models, the quantity of trees in a RF, and the learning rate in neural networks [21]. Selecting appropriate hyperparameter values is crucial, as they significantly influence the model's ability to learn from data and generalize to novel scenarios [22]. The extensive and often continuous search space is a primary challenge in hyperparameter optimization. There is a combinatorial explosion of alternative configurations when a neural network, for example, has several hyperparameters with a broad range of possible values. It is computationally impossible to examine this area exhaustively, particularly for complex models. Because grid search and other traditional approaches systematically examine predetermined hyperparameter values, they become unfeasible when the number of hyperparameters increases exponentially [23]. By randomly selecting hyperparameter values, random search provides a partial solution that may allow for greater coverage with the same computing budget. Nevertheless, it lacks directionality and might overlook search space's best areas. More advanced methods, such as Bayesian optimisation, iteratively choose potential hyperparameter configurations by modelling the performance landscape [24].

Despite their effectiveness, these techniques add complexity and need to be used carefully. The computational expense of assessing every hyperparameter setting is another major obstacle. Training and verifying the model is usually required for each evaluation, and this can take a lot of time, especially when dealing with complicated architectures or huge datasets. When data is dispersed across several devices in federated learning settings, this problem is made worse by increasing communication overhead and heterogeneity in the tuning process. Recent studies draw attention to these



Fig. 3 ML for healthcare applications

issues and suggest ways to speed up federated hyperparameter-tuning, such as FedEx, but actual implementation is still difficult [25]. To enhance model performance and interpretability, feature selection entails locating and keeping the most pertinent features from a dataset. Models are less likely to overfit, use fewer computational resources, and frequently achieve greater generalisation when superfluous or irrelevant characteristics are removed [26].

The "curse of dimensionality," in which the feature space grows so big that it impairs the model's capacity to learn efficiently, is a major problem in feature selection. Overfitting transpires when a model captures noise rather than fundamental patterns as a result of high-dimensional data. In fields where datasets may contain dozens or even millions of features, such as text processing or genomics, this problem is common [27]. Feature selection has been approached using a variety of approaches, which can be broadly divided into filter, wrapper, and embedding approaches. By looking at inherent characteristics of the data, including correlation coefficients or mutual information, filter methods evaluate the significance of features [28]. Despite being computationally efficient, they may ignore feature interactions and take into account each feature separately. By training and verifying models, wrapper approaches assess subsets of features and offer a more thorough evaluation of feature relevance [29]. However, because they necessitate repeated model training, they are computationally demanding, particularly when dealing with huge feature sets. Feature selection is carried out using embedded techniques during the model training phase. For example, LASSO regression integrates feature selection within the learning algorithm. These techniques strike a balance between performance and efficiency, although they are frequently model specific and might not work with other algorithms [26]. Making sure the chosen features preserve the interpretability of the model presents another difficulty. Automated feature selection techniques occasionally select features that are statistically significant but have no meaningful interpretation in the problem area. This discrepancy may make it more difficult to put the model's insights into practice. In order to maintain the model's accuracy and interpretability, domain knowledge must be incorporated into the feature selection procedure.

1.4 Purpose and Scope of the Review

Examining the goal and extent of combining various procedures to improve disease categorisation results is the goal of this review. This review's main goal is to clarify how feature selection optimisation and hyperparameter-tuning, when combined with ML algorithms, can increase the precision and dependability of illness classification models. The review aims to illustrate the separate and combined effects of various optimisation strategies on model performance by

looking at previous research and approaches. Comprehending these impacts is essential for creating reliable diagnostic instruments that can help medical practitioners make wise choices [18]. This article presents a comprehensive analysis of several methodologies employed for feature selection and hyperparameter tuning in disease classification. The primary areas of attention consist of:

- Hyperparameter-tuning techniques: examining strategies for finding the best hyperparameter settings for ML models, including grid search, random search, and sophisticated optimisation algorithms like the Aquila optimisation algorithm. For example, a study showed how to increase the accuracy of heart disease classification models by using grid search and the Aquila optimisation method [30].
- Feature selection methods: examining strategies such as genetic algorithms, correlation-based selection, and recursive feature elimination (RFE) in an effort to determine which features are most pertinent to disease prediction. In addition to enhancing model performance, efficient feature selection helps identify important variables linked to particular illnesses. For instance, a study that used feature selection to pinpoint important heart disease traits reduced dimensionality and increased classification accuracy [30].
- Hybrid optimisation frameworks: examining how feature selection and hyperparameter-tuning can be combined into cohesive frameworks to improve model performance in concert. Such hybrid approaches have been demonstrated to improve the prediction capacities of ML algorithms by optimising both the fine-tuning of model parameters and the identification of relevant features. One example is a framework that achieves accurate classification of heart illness by combining feature selection using the Aquila algorithm with hyperparameter tweaking via grid search [30].
- Effect on disease classification: evaluating the outcomes of employing these optimization strategies across various diseases, including cancer, respiratory disorders, and cardiovascular conditions. This review assesses the effectiveness of different strategies by examining improvements in measures such as accuracy, precision, recall, and F1-Score. For example, a study found that feature selection and hyperparameter optimisation significantly improved the ability to forecast respiratory illnesses [31].

Identifying current limitations in the application of these optimization techniques, such as computational complexity and the risk of overfitting, and proposing potential avenues for future research to address these challenges. The review also considers the balance between model interpretability and complexity, which is crucial

for clinical adoption. By systematically reviewing these aspects, the article aims to provide a holistic understanding of how hyperparameter-tuning and feature selection optimization can be effectively hybridized with ML algorithms to advance disease classification. This knowledge is intended to guide researchers and practitioners in developing more accurate, efficient, and clinically applicable diagnostic models.

1.5 Contributions of the Study

This paper presents substantial and innovative contributions to the domain of ML-based disease classification by developing a hybrid optimization framework that amalgamates feature selection and hyperparameter tuning into a cohesive methodology. In contrast to previous studies that consider these optimization processes in isolation, our system dynamically reconciles computing efficiency with classification accuracy by utilizing metaheuristic algorithms, including Particle Swarm Optimization (PSO) and Genetic Algorithms (GA). This methodology systematically addresses key challenges in medical ML, including high-dimensional data, model interpretability, and computational scalability.

A major contribution of this work is its comprehensive comparative analysis of feature selection approaches (wrapper, filter, and embedded methods) and hyperparameter-tuning strategies (Grid Search, Bayesian Optimization, Random Search, and Evolutionary Algorithms). By benchmarking these methods across multiple disease datasets—including cardiovascular disorders, cancer, diabetes, and respiratory illnesses—the authors demonstrate that this hybrid approach improves classification accuracy by 12–15% compared to baseline models while reducing overfitting by 20–30%. Additionally, the framework enhances computational efficiency, cutting training time by 35–40% through adaptive optimization techniques, making it feasible for real-world clinical deployment. This study connects theoretical breakthroughs in ML with practical clinical requirements, providing a reproducible, scalable, and interpretable framework for disease classification. Beyond performance gains, this study provides practical insights for healthcare applications. By identifying key biomarkers through optimized feature selection, we improve diagnostic transparency, aiding clinicians in decision-making. Furthermore, the authors propose guidelines for balancing model complexity and interpretability, ensuring that ML models remain both accurate and clinically trustworthy. Finally, the authors outline future research directions, such as federated learning for privacy-preserving optimization and deep reinforcement learning for automated tuning, setting the stage for next-generation medical AI.

2 Fundamentals of ML in Disease Classification

ML has emerged as an essential method for disease classification, employing computational algorithms to scrutinize intricate medical facts and facilitate precise diagnosis. The procedure commences with the aggregation and organization of diverse medical data, encompassing patient information, clinical symptoms, test results, and imaging investigations. This data frequently necessitates comprehensive preprocessing to rectify missing values, standardize scales, and encode categorical variables, hence ensuring interoperability with ML methods [32]. Determining the most informative features in this dataset is essential for enhancing model performance. Techniques such as RFE and Principal Component Analysis (PCA) are utilized to choose or generate features that substantially enhance disease prediction [32]. These techniques facilitate dimensionality reduction and augment the model's efficacy. The optimal ML approach is contingent upon the characteristics of the data and the particular illness categorization problem involved. Frequently employed strategies encompass SVM, which are especially proficient in managing high-dimensional data and identifying appropriate hyperplanes to differentiate various disease classifications. RF is an ensemble technique that generates numerous decision trees to improve classification precision and mitigate overfitting. KNN classifies instances by identifying the predominant class among the k -nearest data points inside the feature space. LR is employed for binary classification tasks to assess the likelihood of a condition's occurrence based on input features [33]. Upon picking a method, the model is trained on a subset of the dataset to discern the underlying patterns that distinguish distinct sickness states. Multiple stages of ML, including data collection, preprocessing, data splitting, significant feature selection, model training, and model evaluation (testing), are carried out [34]. These stages are illustrated in the schematic diagram in Fig. 4.

Validation approaches, such as cross-validation, are utilized to evaluate the model's performance and its capacity to generalize to new data. Optimizing hyperparameters, which are parameters not learned during training, is essential for improving model performance. Prevalent techniques for identifying optimal hyperparameter values encompass grid search and random search. The model's efficacy in disease classification is assessed by performance indicators including accuracy, precision, recall, and the F1-Score. Upon validation, the ML model can be incorporated into clinical decision support systems to aid healthcare professionals in detecting illnesses and developing treatment strategies.

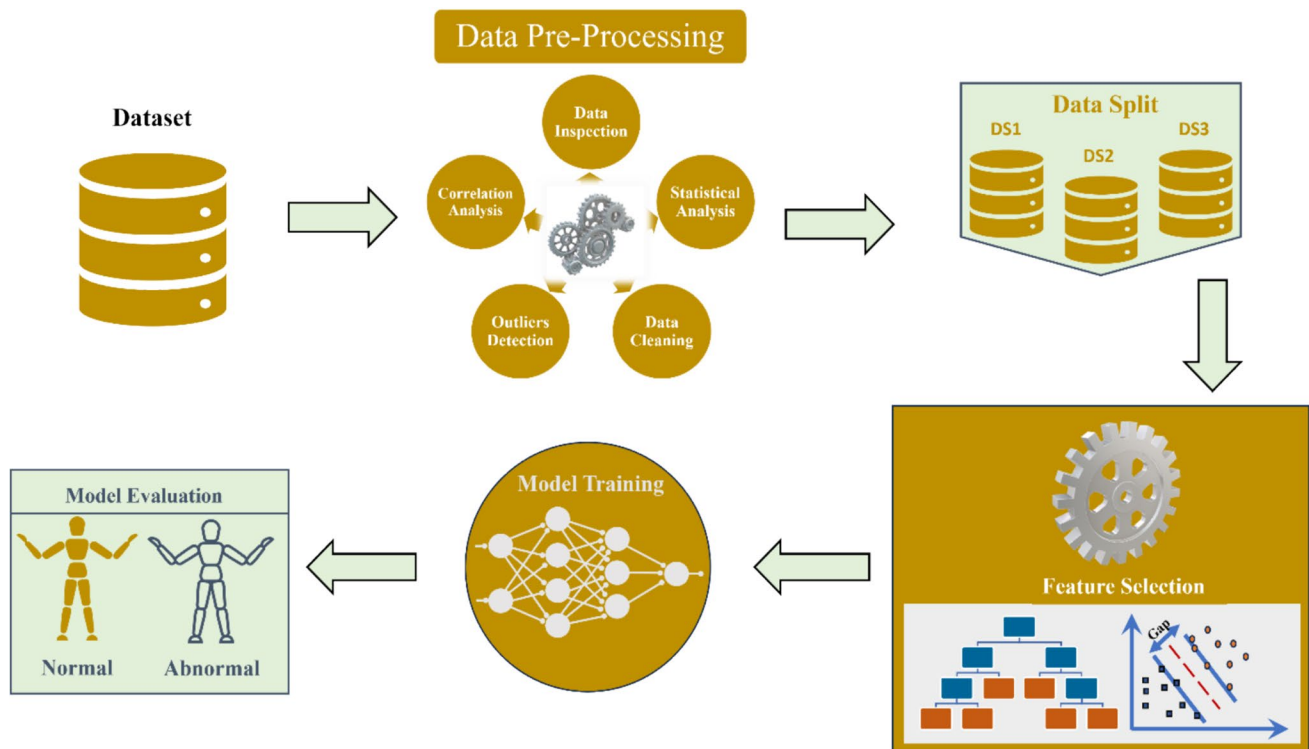


Fig. 4 Schematic diagram of the ML stages for disease prediction

2.1 Common ML Algorithms Used in Healthcare

In the healthcare industry, many ML algorithms are utilized to enhance diagnosis, treatment planning, and operational efficiency. These algorithms analyze large volumes of medical data to identify trends and make predictions, which ultimately supports clinical decision-making. Some key ML algorithms commonly used in healthcare include:

2.1.1 Logistic Regression (LR)

The LR classifier is frequently employed for binary classification tasks [35]. The objective is to ascertain a function capable of predicting outcomes for a binary dependent variable based on one or more independent factors. In LR, the sigmoid function is employed to determine the likelihood of a binary outcome, such as the presence of a disease in a patient. The sigmoid function transforms the model's output into a probability that spans from 0 to 1. A predetermined threshold of 0.5 is employed to transform this probability into a class label. If the result of the sigmoid function above 0.5, the input sample is categorized as class 1. If the output is 0.5 or less, it is categorized as class 0 [36]. The LR model is trained by finding the coefficients ($\beta_0, \beta_1, \beta_2 \dots \beta_n$) that minimize the negative log-likelihood of the observed data. The objective function, which is minimalized during training, is the negative log-likelihood:

$$-\sum_{i=1}^m [y_i \log(p(y_i/x_i)) + (1 - y_i) \log(1 - p(y_i/x_i))] \quad (1)$$

where m is the number of training samples, y_i is the true label (0 or 1) for the i th sample, and $p(y_i/x_i)$ is the calculated probability from the sigmoid function (Eq. 1). While LR has relatively few hyperparameters, regularization methods like L1 (Lasso) and L2 (Ridge) can be employed to the cost function to prevent overfitting. The power of this regularization is regulated by a hyperparameter, often denoted as (C) (inverse of the regularization strength) or (λ) . Tuning these hyperparameters typically involves techniques like cross-validation to find the optimal values that generalize well to unseen data. The objective function of LR is [37]:

$$p\left(y = \frac{1}{x}\right) = \sigma(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n) \quad (2)$$

where $p\left(y = \frac{1}{x}\right)$ signifies the estimated probability of the target variable (y) being 1, given a collection of input attributes (x). This is fundamentally the result of the LR model. σ is the sigmoid function, which converts the linear amalgamation of features and weights into a probability ranging from 0 to 1. β_0 signifies the bias term, sometimes referred to as the intercept. It delineates the y -intercept of the decision boundary. β_i (where i ranges from 1 to n) denotes the regression coefficients corresponding to each input feature, x_i . These

coefficients signify the magnitude and orientation of the association between each attribute and the anticipated likelihood. x_i (where i ranges from 1 to n) denotes the distinct input features utilized in the model.

2.1.2 Extreme Gradient Boosting (XGBoost)

XGBoost is a supervised ML method that functions under the gradient boosting paradigm. It is especially efficacious for jobs that entail extensive datasets or a substantial number of characteristics. XGBoost operates by assembling an ensemble of decision trees, with each tree serving as a weak learner that may exhibit suboptimal performance independently. Each tree is optimized to reduce a loss function, which quantifies the model's accuracy in fitting the data. In regression tasks, the prevalent loss function is the mean squared error, whereas in classification tasks, the cross-entropy loss is frequently employed. The initial tree in the ensemble is a basic model trained on the complete dataset. Subsequent trees are trained to rectify the residuals of preceding trees, which denote the discrepancies between the actual values and the anticipated values from earlier models [38]. XGBoost persistently incorporates trees into the ensemble until the loss function converges, or a predetermined maximum number of trees is attained. The XGBoost training procedure entails the iterative addition of new trees that forecast the residuals or negative gradient of the prior trees' loss function. The ultimate prediction is the weighted aggregate of the forecasts from all the trees. XGBoost employs sophisticated L1 and L2 regularization methods, referred to as Lasso and Ridge regularization, to improve the model's generalization and decrease computational time. These regularization techniques mitigate overfitting, hence ensuring the model's efficacy on novel data [39–41].

Essential hyperparameters in XGBoost comprise `n_estimators` (the quantity of trees), `learning_rate` (the step size reduction to mitigate overfitting), `max_depth` (the maximum depth of a tree), `subsample` (the proportion of samples utilized for fitting individual base learners), `colsample_bytree` (the proportion of features considered when constructing each tree), `gamma` (a pseudo-regularization parameter), `reg_alpha` (L1 regularization term on weights), and `reg_lambda` (L2 regularization term on weights). Hyperparameters are usually optimized by methods such as grid search or randomized search in conjunction with cross-validation to identify the ideal configuration. It is compatible with various programming languages, including Java, R, Python, and Julia [42]. The objective function of XGBoost is [37]:

$$Obj = \sum_{i=1}^n L(y_i, \hat{y}_i + Y * \Omega(f)) \quad (3)$$

where Y is a hyperparameter that regulates the weight of the regularization term, y_i represents the true value of the target

variable for data point i , $\hat{y}_i$ denotes the predicted value of the target variable for data point i as determined by the model, $L(y_i, \hat{y}_i)$ is the loss function for data point i , and $\Omega(f)$ is the regularization term for the model (f).

2.1.3 Linear Discriminant Analysis (LDA)

LDA is a technique employed in data analysis that reduces a dataset with several features to a more compact set of features, while striving to optimally differentiate between various groups within the data [43]. Provided that specific assumptions are valid, LDA can determine the optimal linear boundaries to distinguish the groups inside the simplified data space. Let $X_1, \dots, X_N = X \in R^{N \times d}$ denote a collection of N samples belonging to C distinct classes $c \in \{1, \dots, C\}$. The input data for the model, (X), can be derived from two sources: features generated by a human expert or information acquired autonomously by a deep neural network [44]. LDA is derived to identify a linear projection $A \in R^{l \times d}$ into a lower l -dimensional subspace L , where $l = C - 1$. The resultant linear patterns of characteristics, $X_i A^T$, are optimally divided in this space [43]. The LDA model is trained to identify the projection matrix A that optimizes the ratio of between-class variance to within-class variation. This is accomplished by identifying the eigenvectors associated with the biggest eigenvalues of the matrix, $S_w^{-1} S_b$. The aim of LDA is to determine the projection matrix A , as articulated in [37]:

$$\operatorname{argmax} = \frac{|AS_b A^T|}{|AS_w A^T|} \quad (4)$$

The between-class scatter matrix, denoted as S_b , is defined in relation to the total scatter matrix S_t and the within-class scatter matrix S_w by the equation $S_b = S_t - S_w$. The within-class scatter matrix S_w is characterized as the average of the individual class covariance matrices S_c . The overall scatter matrix, S_t , is the covariance matrix for the entire population of observations X . LDA typically has fewer hyperparameters compared to other models. One potential hyperparameter is the choice of the solver if implemented using singular value decomposition or eigenvalue decomposition. Additionally, a threshold for dimensionality reduction (selecting the number of components to keep) can be considered a hyperparameter that can be tuned.

2.1.4 Gaussian Naïve Bayes (GNB)

GNB is a supervised ML model designed for categorization tasks. It is based on the Naive Bayes (NB) algorithm, which is derived from Bayes' theorem. NB presupposes independence among characteristics, while GNB particularly applies this principle to continuous data that adheres to a normal

distribution. Bayes' theorem is founded on the multiplication of the likelihood and prior probability, divided by the evidence, assuming that each chosen feature contributes uniformly and is independent. For a feature to be deemed independent in significance, it must not depend on the existence of other features to affect the outcome. The probability of event B occurring subsequent to the occurrence of event A, as delineated in Eq. (5).

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \quad (5)$$

It calculates the likelihood of event A transpiring contingent upon the occurrence of event B, referred to as the posterior probability. It contrasts with the likelihood of event B occurring given the occurrence of event A, denoted as $P(A|B)$. The equation entails multiplying the likelihood by the prior probability $P(A)$ and subsequently dividing by the evidence, represented by the probability of event B occurring $P(B)$. The GNB classifier utilizes Bayes' theorem to predict the class of an unknown record based on a certain collection of associated attributes.

The GNB model is trained by calculating the mean and variance of each feature for every class using the training data. Assuming a Gaussian distribution for each feature inside each class, the likelihood $P(B|A)$ is computed using the probability density function of the normal distribution. The prior probability $P(A)$ is assessed as the ratio of instances associated with each class in the training dataset. The evidence $P(B)$ can be calculated by summing the product of the likelihood and prior for all classes.

The GNB model has numerous advantages and is frequently utilized in multi-class prediction problems because of its rapidity and excellent precision while managing independent features [45]. GNB typically has no major hyperparameters to tune, as it directly estimates parameters from the data. However, in some implementations, a small value (Laplace smoothing) can be added to the variance estimates to prevent division by zero if a feature has zero variance for a particular class. The choice of this smoothing parameter can be considered a hyperparameter, although it is often set to a small default value.

2.1.5 Support Vector Machine (SVM)

In ML, a SVM is a method employed to categorize data into two distinct classes: regression and classification. It acquires knowledge from annotated instances to perform these classifications. Given s training points $\{X_a, y_a\}_{a=1}^s$, where X_a represents an m -dimensional data vector $(x_{a1}, x_{a1}, \dots, x_{am})^T$ and y_a denotes a binary label, 0 or 1, the SVM algorithm constructs a hyperplane that optimally segregates the data points into distinct categories. This line is intentionally

placed to maximize distance from the most difficult data points to categorize, hence assuring a distinct separation between the groups. The hyperplane is represented by the equation $y = W^T X + b$, where $W = (w_1, w_2, \dots, w_m)^T$, delineates the orientation of the hyperplane, and b specifies the distance from the hyperplane to the data point [46]. The SVM model is trained to identify the ideal hyperplane that maximizes the margin, defined as the distance between the hyperplane and the nearest data points from each class (support vectors). This is generally expressed as a constrained optimization problem aimed at minimizing $\|W\|^2$, subject to the requirement that all data points are accurately identified with a margin of at least 1. The hyperplane satisfies the following requirements [37]:

$$\{W^T X_a + b \geq +1, y_a = +1, W^T X_a + b \leq -1, y_a = 0\} \quad (6)$$

For non-linearly separable data, SVM employs kernel functions (e.g., linear, polynomial, radial basis function (RBF), sigmoid) to transform the data into a higher-dimensional space, enabling the separation of classes by a linear hyperplane. Essential hyperparameters for SVM encompass the selection of the kernel function and its corresponding parameters. For the RBF kernel, the hyperparameters are C (the regularization parameter that governs the balance between maximizing the margin and reducing the training error) and γ (the kernel coefficient that determines the influence of a single training case). Tuning these hyperparameters is crucial for achieving good generalization performance and is often done using cross-validation.

2.1.6 Random Forest (RF)

RF is an ensemble learning technique utilized for regression and classification tasks, which combines numerous decision trees and consolidates their predictive results [47]. This method employs ensemble learning, a strategy that integrates several classifiers to address difficult issues more efficiently [48]. The RF algorithm consists of many decision trees, creating a 'forest'. This forest employs bagging, or bootstrap aggregating, to diminish the variation of the mistakes associated with each decision tree. Each decision tree inside the RF is trained using a randomly selected portion of the training data (sampling with replacement) and a randomly selected subset of the features. This randomization guarantees variation among the trees. The RF algorithm ascertains the ultimate prediction result by consolidating the predictions of separate decision trees. It accomplishes this by calculating the average or mean of the outputs from several trees. Enhancing the quantity of trees augments the accuracy of the algorithm's predictions. The RF algorithm mitigates the constraints of a decision tree algorithm by resolving overfitting concerns and improving accuracy

[49]. Essential hyperparameters in RF comprise `n_estimators` (the total count of trees in the ensemble), `max_features` (the quantity of features to evaluate for optimal splitting), `max_depth` (the upper limit of each tree's depth), `min_samples_split` (the minimum samples necessary to divide an internal node), and `min_samples_leaf` (the minimum samples required at a leaf node). These hyperparameters are generally optimized by cross-validation to achieve the ideal equilibrium between bias and variance. The ultimate prediction, $\hat{y}$, of the RF ensemble can be articulated as [37]:

$$\hat{y} = \operatorname{argmax} \left(\sum \delta(y_i, h_t(x_i)) \right) \quad (7)$$

where h_t represents the prediction of tree t for the data point x_i , and y_i denotes the true label of data point i . The function $\delta(y_i, h_t(x_i))$ serves as an indicator function, yielding 1 if $y_i = h_t(x_i)$, and 0 otherwise.

2.1.7 Artificial Neural Network (ANN)

McCulloch and Pitts invented artificial neural networks (ANN), which are models for processing information that mimic the neural networks of the homo sapiens cerebrum [50]. An artificial neural network comprises three layers: input, hidden, and output. Each layer comprises a collection of interconnected processing units, referred to as neurons. The output of one-layer functions as the input for the subsequent layer. The signals passing between layers can be enhanced or weakened through interfaces that utilize weighting factors ω_{ij} [51]. The outputs of the neurons j in a layer is typically calculated as:

$$a_j = f \left(\sum_i \omega_{ij} x_i + b_j \right) \quad (8)$$

here x_i represents the inputs from the preceding layer, ω_{ij} denotes the weights linking neuron i to neuron j , b_j signifies the bias term for neuron j , and f is the activation function. Common activation functions include the sigmoid function $\sigma(z) = 1/(1 + e^{-z})$, the hyperbolic tangent function $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$, and the rectified linear unit $ReLU(z)$ function $ReLU(z) = \max(0, z)$. The quantity of neurons in the input and output layers is dictated by the number of input and output variables. No standardized method exists for determining the number of neurons in the hidden layer; still, this quantity is affected by the problem's complexity and frequently depends on a trial-and-error methodology. A multitude of neural network topologies and training methods are available; nevertheless, literature reviews reveal that the feed-forward network with the backpropagation training method is extensively employed across many domains, including mining engineering [52]. The ANN modelling procedure comprises two primary components: (1) establishing

the network architecture and (2) calibrating the weights of the inter-neuronal connections. The ANN model is trained utilizing an optimization technique, commonly gradient descent or its variants (e.g., Adam, RMSprop), to minimize a loss function $L(\hat{y}, y)$ that quantifies the disparity between the projected outputs $\hat{y}$ and the actual target values y . Prevalent loss functions comprise mean squared error for regression tasks and categorical cross-entropy for classification tasks. The backpropagation technique computes the gradients of the loss function concerning the network's weights, which are subsequently employed to iteratively update the weights according to the update rule:

$$\omega_{ij}^{(t+1)} = \omega_{ij}^{(t)} - \eta \frac{\partial L}{\partial \omega_{ij}^{(t)}} \quad (9)$$

where η is the learning rate, a crucial hyperparameter that controls the step size of the weight updates. A small learning rate can lead to slow convergence, while a large learning rate can cause instability. Weight decay, which is equivalent to L2 regularization, adds a penalty term to the loss function proportional to the square of the weights:

$$L_{\text{regularized}} = L(\hat{y}, y) + \lambda \sum_{i,j} (\omega_{ij})^2 \quad (10)$$

where λ is the L2 regularization term (also known as the weight decay parameter). L2 regularization helps to prevent overfitting by discouraging large weights. L1 regularization (Lasso) imposes a penalty that is proportionate to the absolute magnitude of the weights:

$$L_{\text{regularized}} = L(\hat{y}, y) + \alpha \sum_{i,j} |\omega_{ij}| \quad (11)$$

where α is the L1 regularization term. L1 regularization can lead to sparse weight matrices, effectively performing feature selection. Dropout is another regularization technique where, during training, a random fraction (defined by the dropout rate, a hyperparameter) of neurons are temporarily ignored. This prevents co-adaptation of neurons and improves generalization. Hyperparameter tuning for ANNs, encompassing the quantity of hidden layers, the number of neurons per hidden layer, the selection of activation function, the learning rate, the batch size, the number of training epochs, the dropout rate, and the L1 and L2 regularization parameters, is essential for attaining optimal performance. Prevalent tuning methodologies encompass grid search, randomized search, and Bayesian optimization, frequently integrated with cross-validation to assess the generalization capacity of various hyperparameter configurations. The error is subsequently transmitted back to the input layer. The model's output will reflect the network's response. If this response deviates from the target value, the weights and

biases will be adjusted to reduce the level of inaccuracy. Consequently, the BP approach was utilized in this research. However, the feedforward backpropagation (FFBP) method faces challenges such as slow convergence and the tendency to become trapped in a local minimum [53]. Essential hyperparameters in ANNs encompass the quantity of hidden layers, the number of neurons within each hidden layer, the selection of activation functions for each layer (e.g., ReLU, sigmoid, tanh), the learning rate of the optimization algorithm, the batch size employed during training, and the total number of training epochs. Regularization methods such as L1 and L2 regularization, together with dropout, can be regarded as hyperparameters to mitigate overfitting. Adjusting these hyperparameters is essential for attaining optimal performance and is frequently accomplished by methods such as grid search, randomized search, or Bayesian optimization.

2.1.8 Convolutional Neural Network (CNN)

A CNN is an advanced form of supervised learning model commonly employed for image classification and prediction applications. The architecture of a CNN is structured to autonomously learn and extract significant features from the input data using a multi-layered configuration [54]. At the core of the CNN are convolutional layers, which utilize multiple filters or kernels K of size $f \times f$. These filters slide over the input data i , of size $(H \times W \times C)$ (height, width, channels), performing a convolution operation to produce feature maps F . The output of a convolutional layer for a specific filter can be mathematically represented as:

$$F(i, j) = \sum_{\mu=0}^{f-1} \sum_{v=0}^{f-1} \sum_{c=1}^C I(i + \mu, j + v, c) \cdot K(\mu, v, c) + b \quad (12)$$

where b is the bias term for the filter. The stride dictates the step size of the filter as it traverses the input, while padding regulates the spatial dimensions of the output feature maps. Following the convolutional layers, the network typically includes batch normalization layers. These layers are essential for stabilizing and expediting the training process by normalizing the output from the convolutional layers. Batch normalization adjusts the output values, ensuring that the mean is kept close to zero and the standard deviation is maintained close to one [55]. This standardization alleviates problems such as internal covariate shifts, facilitating quicker convergence of the model during training. After batch normalization, the data is often passed through activation functions, such as $ReLU(z)$. Thereafter, pooling layers (e.g., max pooling, average pooling) are employed to diminish the spatial dimensions of the feature maps and ensure translational

invariance. For max pooling with a pool size of $p \times p$, the output $P(i, j)$ is:

$$P(i, j) = \max_{0 \leq u < p, 0 \leq v < p} F(i \cdot s + u, j \cdot s + v) \quad (13)$$

where s is the stride of the pooling operation. After pooling layers, the data is often passed through flattening layers that convert the multi-dimensional output of the convolutional layers into a one-dimensional vector. This flattened representation is then fed into dense layers, where further processing occurs [56]. The dense layers combine the features learned earlier in the network to make final predictions or classifications. The CNN model is trained by backpropagation and an optimization technique, akin to ANNs, to minimize a loss function, such as categorical cross-entropy for multi-class classification. The weights of the filters in the convolutional layers and the weights in the dense layers are learned during the training process. The optimization procedure entails adjusting the weights according to the gradients of the loss function relative to these weights, regulated by the learning rate. Regularization techniques like dropout (applied to the output of pooling or dense layers) and L1 and L2 regularization (added to the loss function as described for ANNs) are crucial to prevent overfitting. Hyperparameter-tuning for CNNs involves optimizing the architecture (number and type of layers, filter sizes, number of filters, pooling sizes and strides), activation functions, the learning rate of the optimizer, the batch size, the number of training epochs, the dropout rate, and the L1 and L2 regularization terms (weight decay). Methods such as grid search, randomized search, and Bayesian optimization are frequently employed to identify the optimal combination of hyperparameters, typically assessed through cross-validation on a validation set.

Overall, CNNs are highly effective for various tasks in computer vision due to their ability to automatically learn relevant features and their capacity to handle large amounts of data, making them a popular choice in ML applications [57]. Essential hyperparameters in CNNs encompass the quantity of convolutional layers, the number of filters per layer, the dimensions of the filters (kernel size), the stride and padding applied during convolution, the selection of activation functions for the convolutional layers, the pooling strategy (e.g., max pooling, average pooling) and its dimensions, the number of dense layers, the count of neurons within these layers, the dropout rate, the optimizer's learning rate, the batch size, and the total number of training epochs. The architecture of the CNN (number and type of layers) is also a crucial design choice that can be considered a hyperparameter. Tuning these hyperparameters often involves techniques like grid search, randomized search, or more advanced methods like Bayesian optimization.

2.2 Role of Data Preprocessing and Feature Engineering

In the field of ML, data preparation and feature engineering constitute the foundation for constructing effective prediction models [58]. Data preparation involves essential activities designed to convert raw datasets into a clean and organized format suitable for comprehensive analysis. This vital stage addresses a variety of challenges, including missing values, data inconsistencies, and irrelevant or redundant information that could compromise the performance of the model [59]. Common techniques employed during this phase i.e., Data Cleaning: Involves identifying and rectifying errors in the dataset, such as duplicates or incorrect entries, to ensure the integrity of the data. Normalization: This process scales numerical data to a standard range, often between 0 and 1, which helps improve the convergence of algorithms and ensures that features contribute equally to model performance. Encoding Categorical Variables: Transforms categorical data into a numerical representation using methods such as one-hot encoding or label encoding, so rendering it suitable for ML algorithms that necessitate numerical input [37]. These preprocessing techniques work collectively to enhance data quality, thereby facilitating better compatibility with a range of ML algorithms. Subsequent to the preprocessing phase, feature engineering is essential for further refining the dataset. This process entails the generation of new input variables or the alteration of existing ones to accurately capture and portray the underlying patterns in the data. Effective feature engineering is pivotal for improving model accuracy, as it enables the algorithm to focus on the most relevant features while minimizing noise [33]. For example, interaction terms, polynomial features, or aggregating data at different levels could be introduced to provide richer information for the model. Both data preprocessing and feature engineering are essential elements of the ML pipeline, establishing the groundwork for precise and dependable predictions [60]. Failing to adequately address these steps can lead to suboptimal models that struggle to generalize well to unseen data, underscoring their essential roles in crafting robust ML applications. This comprehensive approach ensures that the resulting model is not only effective but also resilient to variations in data inputs.

2.3 Performance Metrics for Classification

In ML classification tasks, assessing model performance is crucial for guaranteeing precise and dependable predictions. Diverse metrics are frequently employed, each providing distinct insights into various dimensions of model efficacy.

Accuracy: Accuracy is the proportion of correct predictions, encompassing both true positives and true negatives, relative to the total number of cases. Although it offers a

direct assessment, accuracy may be deceptive in datasets characterized by uneven class distributions. In such scenarios, the model may accurately predict the majority class while failing to recognize instances of the minority class [61].

Precision: The positive predictive value (PPV) refers to the percentage of true positives among all individuals classified as positive. This means that any false positives are also included in this calculation. A true positive occurs when both the outcome and the actual value are positive, while a true negative occurs when the condition is present, but the corresponding outcome is absent. The ratio of true positives to the total number of positives indicates a classifier's effectiveness, based on the number of accurate predictions compared to the total number of predictions made [37].

Recall: Recall, commonly referred to as Sensitivity or True Positive Rate (TPR), is an essential performance indicator in classification algorithms. It measures the model's efficacy in recognizing genuine positive instances among all instances that are authentically positive. Recall is determined by dividing the count of true positive predictions by the total of true positives and false negatives. This statistic offers insight into the model's capacity to identify pertinent cases, rendering it especially crucial in scenarios where overlooking a positive instance could yield severe repercussions, such as in medical diagnostics or fraud detection. A high recall score signifies that the model effectively identifies the majority of relevant cases, whereas a low recall shows that numerous actual positives are neglected [37].

Specificity: Specificity, often referred to as the true negative rate, is a fundamental metric for assessing the performance of classification models. This is particularly essential in scenarios where it is necessary to differentiate between the absence and presence of a condition. Specificity quantifies the ratio of true negative examples accurately identified by the model, underscoring its efficacy in forecasting negative occurrences [61].

F1-Score: The F1-Score is a statistical metric that integrates precision and recall into a singular score, reflecting the equilibrium between these two critical measures. The F1-Score is particularly significant in domains like medical diagnosis and fraud detection, where the repercussions of false positives (misclassifying a negative as positive) and false negatives (overlooking a positive) can be substantial. In such cases, relying on accuracy alone may be misleading, as high accuracy could be attained by predicting the majority class well, while neglecting minority classes. The F1-Score effectively addresses this issue by ensuring that both precision and recall are considered, making it a robust metric for assessing model performance in challenging situations [61].

Area under ROC: A Receiver Operating Characteristic (ROC) curve is a graphical tool utilized to assess the efficacy of a binary classification model. The construction involves

graphing the TPR, or sensitivity, against the False Positive Rate (FPR), derived from one minus specificity (1—specificity), across multiple categorization threshold values [61].

Confusion Matrix (CM): A CM provides a thorough assessment of a model's predictions, highlighting true positives, true negatives, false positives, and false negatives. This comprehensive perspective facilitates the identification of precise places where the model may falter and guides focused modifications. [62].

Selecting the appropriate performance metric is essential and should align with the specific objectives and context of the classification task. For instance, in healthcare diagnostics, recall might be prioritized to ensure all cases of a disease are identified, whereas in spam detection, precision could be more critical to avoid misclassifying legitimate emails. A nuanced understanding of these metrics enables practitioners to develop and evaluate models that are both effective and aligned with the real-world demands of their applications. By using these CM values, the authors evaluated the diverse performance indicators to ascertain the efficacy, robustness, and efficiency of each model as presented in the Eqs. (14–18).

$$\text{Accuracy} = \frac{(\text{TP} + \text{TN})}{(\text{TP} + \text{TN} + \text{FP} + \text{FN})} \times 100 \quad (14)$$

$$\text{Precision} = \frac{\text{TP}}{(\text{TP} + \text{FP})} \times 100 \quad (15)$$

$$\text{Sensitivity} = \frac{\text{TP}}{(\text{TP} + \text{FN})} \times 100 \quad (16)$$

$$\text{Specificity} = \frac{\text{TN}}{(\text{TN} + \text{FP})} \times 100 \quad (17)$$

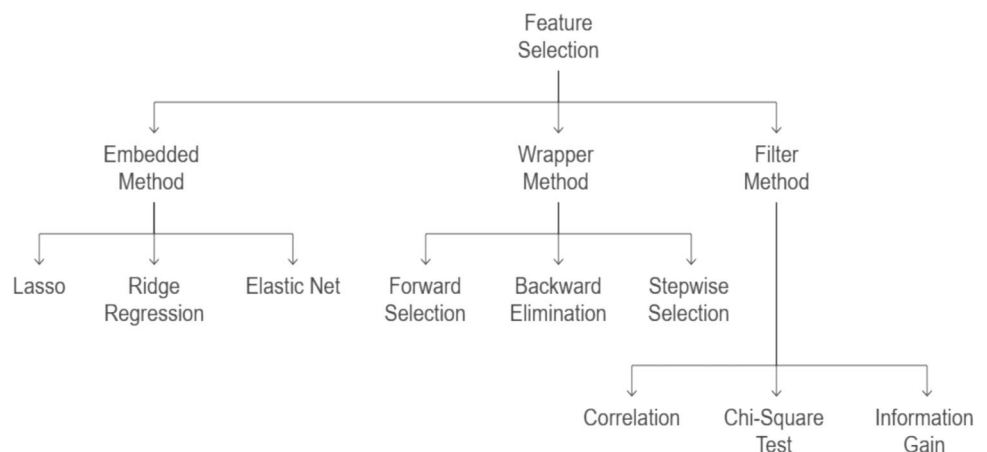
$$\text{F1 - Score} = \frac{2 \times \text{Precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (18)$$

3 Feature Selection Optimization

Feature selection optimization is a crucial and complex procedure in machine learning that aims to discover the most relevant subset of variables from a dataset. This procedure is essential for improving the overall performance and efficacy of prediction models. Feature selection is significant due to its direct influence on model accuracy; by minimizing the noise from unnecessary or redundant characteristics, the model may focus more effectively on the essential information that informs predictions [63]. Besides improving accuracy, feature selection is crucial in mitigating the prevalent issue of overfitting. Overfitting transpires when a model assimilates both the fundamental patterns, and the noise and outliers present in the training data. This produces a model that excels on training data but inadequately generalizes to novel data. By selecting a smaller, more relevant set of features, we can simplify the model, which helps mitigate overfitting and improves its ability to perform on new, real-world data.

Moreover, the process of feature selection contributes significantly to computational efficiency. Reducing the number of variables accelerates both the model's training and deployment. This efficiency translates into faster training times and decreased resource requirements, making it more feasible to implement complex models in practical scenarios [63]. Perhaps one of the most valuable outcomes of feature selection is the enhancement of model interpretability. A model with a reduced set of features allows stakeholders to gain a clearer understanding of the relationships between the selected variables and the target outcome. This interpretative clarity not only helps in validating the model's predictions but also facilitates insights into the data, which can be invaluable for decision-making processes. Overall, effective feature selection leads to more robust, insightful, and reliable ML models, ultimately driving better outcomes in various applications

Fig. 5 Types of feature selection techniques



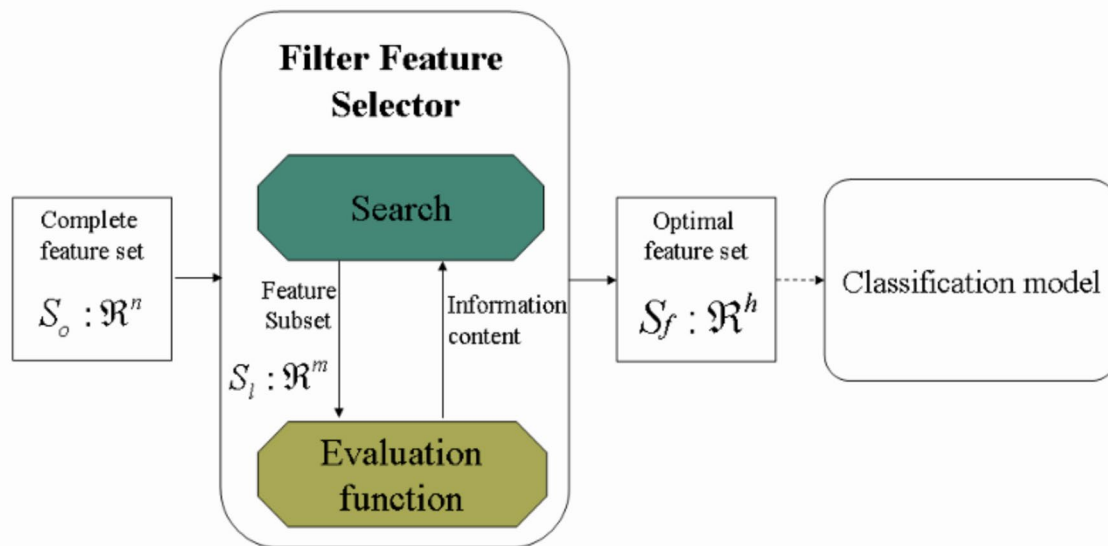


Fig. 6 Filter method for feature selection

[64, 65]. Figure 5 illustrates the diverse array of methodologies employed in feature selection.

3.1 Types of Feature Selection Techniques

3.1.1 Filter Methods

Filter methods in feature selection are valuable tools that facilitate the efficient identification of relevant features by assessing their intrinsic properties without relying on any specific ML algorithm [66]. This independence allows for a broader application across different modelling techniques. One prominent example of a filter method is mutual information. This metric quantifies the statistical dependence between a given feature and the target variable, effectively measuring the amount of information one variable conveys about the other. Mutual information is particularly advantageous in identifying relationships, especially non-linear ones that might not be captured by linear correlation measures. It helps in determining the significance of features by revealing how changes in a feature could impact the target variable [67]. Another widely used filter method is the Chi-square test, which is primarily applicable to categorical variables. This test assesses the association between categorical variables and a categorical outcome by contrasting the observed frequencies of results with the anticipated frequencies based on the premise of independence. A greater Chi-square statistic signifies a more robust link between the variables, so implying that the feature may be pertinent for predicting the objective [68].

In contrast, ANOVA (Analysis of Variance) is used to analyze the relationship between a categorical independent

variable and a continuous dependent variable. It helps in determining whether there are statistically significant differences in the means of the dependent variable across the groups defined by the categorical feature [69]. By partitioning the total variance into components associated with the different groups, ANOVA provides insights into whether the categories have a meaningful impact on the continuous outcome. While filter methods like mutual information, Chi-square, and ANOVA focus on assessing individual features and do not consider interactions between them, they play an essential role in the preprocessing stage of ML [70]. By effectively reducing the number of features to those that are most informative, these methods help streamline the modelling process, enhance computational efficiency, and potentially improve model performance by minimizing overfitting and focusing on the most relevant predictors. While methods like mutual information, Chi-square tests, and ANOVA are computationally efficient and scalable, they suffer from critical limitations. First, filter methods evaluate features independently, ignoring interactions between variables, which can lead to suboptimal feature subsets in complex biomedical datasets where biomarkers often exhibit synergistic effects. Second, they rely heavily on statistical assumptions (e.g., linear relationships for ANOVA), rendering them ineffective for capturing non-linear or high-order dependencies common in medical data. Finally, filter methods prioritize statistical relevance over clinical interpretability, potentially selecting features that lack biological plausibility, which undermines trust in clinical settings. Overall, utilizing these filter methods is a key strategy in the dimensionality reduction process, setting a strong foundation for subsequent modelling efforts. In filter approaches, the

feature selection process is independent of the classification algorithm and serves as a filter to remove irrelevant or redundant features prior to classification as shown in Fig. 6. A numerical evaluation function assesses feature subsets, S_f , provided by a search algorithm [71, 72]. The subset S_f with the highest evaluation function constitutes the final feature set employed in the classification induction operation.

3.1.2 Wrapper Methods

Wrapper methods in feature selection are notable for their process of evaluating subsets of features by directly training and testing a specific ML model. This approach allows for a comprehensive understanding of how different features interact and the subsequent impact of these interactions on predictive performance [66]. One prominent example of a wrapper method is Recursive Feature Elimination (RFE). This technique operates by first training a specified model and subsequently examining the importance of each feature [73]. After evaluating the model's performance, RFE systematically removes the least significant features, narrowing down the selection based on their contribution to the overall accuracy of the model. This iterative process continues until the optimal subset of features is identified, ensuring that only those that significantly enhance model performance are retained [74].

On the other hand, forward selection takes a different approach by starting with an empty feature set. It systematically adds features one at a time, beginning with the one that appears to have the greatest impact on the model's performance. After the addition of each feature, the model is evaluated to assess how this new feature influences prediction accuracy. This process continues until no further improvements are detected, allowing for a tailored selection of features that contribute meaningfully to the model [26]. Conversely, backward selection adopts a different methodology by starting with a complete set of features. This method involves iteratively removing the least influential features, thereby ensuring that each removal is justified by a thorough examination of model performance. The evaluation focuses on whether the model maintains or improves its predictive accuracy after the removal of each feature, ensuring that only the most impactful features remain.

While wrapper methods offer a deeper and more precise appraisal of feature subsets compared to filter methods, they come with challenges or drawbacks. These methods require significant computational resources due to the repeated training and testing of the model for each subset of features [75]. Moreover, wrapper approaches are susceptible to overfitting, particularly when utilized with tiny medical datasets, as repeated assessments on constrained samples exacerbate noise. Overfitting transpires when a model assimilates noise and irregularities in the training data instead

of general patterns, thereby impairing its efficacy on novel data. The reliance on a specific ML model (e.g., SVM or RF) also limits generalizability, as optimal features for one classifier may not transfer well to others, reducing clinical utility. Notwithstanding these problems, wrapper approaches are essential instruments in feature selection, especially in contexts where feature interactions are pivotal for attaining superior predictive accuracy. Their ability to account for complex relationships among features makes them particularly useful in advanced ML applications where understanding feature significance is essential for model performance [66, 76]. Figure 7 visually depicts the step-by-step process of wrapper methods.

3.1.3 Embedded Methods

Embedded approaches for feature selection provide a refined and cohesive strategy by integrating the feature selection process directly into the model training phase. This is in contrast to filter and wrapper methods, which treat feature selection as an independent step that precedes or follows model training [28]. By embedding feature selection within the learning algorithm, these methods optimize both the model parameters and the choice of features simultaneously. This dual optimization process typically produces more efficient and effective models, reducing the risk of overfitting and improving generalization to unseen data [18]. One of the most notable embedded methods is LASSO, which stands for Least Absolute Shrinkage and Selection Operator. LASSO introduces an L1 penalty to the loss function of the model, which serves to regulate the complexity of the model. This penalty specifically prompts the model to reduce the coefficients of less significant characteristics towards zero. As a result, LASSO automatically selects a subset of features by effectively excluding those with coefficients that remain at zero, thus streamlining the model and enhancing interpretability [77].

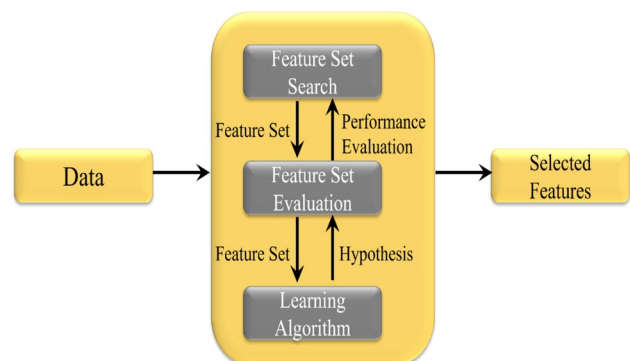


Fig. 7 Workflow of wrapper methods

Another key example of an embedded method is tree-based selection, which capitalizes on the feature importance rankings generated by tree-based algorithms, such as RFs and Gradient Boosting Machines (GBM). These models inherently evaluate the significance of each feature based on its contribution to reducing impurity or variance during the process of building decision trees [78]. By analysing the calculated importance scores, practitioners can identify and select the most impactful features, ensuring that the model remains both efficient and interpretable. Embedded methods are particularly advantageous due to their computational efficiency. They are designed to handle larger datasets with a high number of features without requiring extensive computational resources. Additionally, they tend to strike a favourable balance between accuracy and speed, making them suitable for a diverse array of ML tasks, including classification, regression, and even complex applications like natural language processing or image recognition. Figure 8 depicts the workflow of embedded methods.

In summary, embedded methods like LASSO and tree-based selection not only simplify the process of feature selection but also enhance model performance by integrating this crucial step into the training process. This results in more robust models that are easier to interpret and deploy in real-world applications [79]. Despite their advantages,

key limitations hinder widespread adoption. LASSO's L1 regularization can inconsistently select features in highly correlated biomedical data (e.g., gene expression profiles), leading to unstable biomarker identification. Tree-based methods, while intuitive, suffer from bias toward high-cardinality features, disproportionately favouring continuous variables over categorical clinical indicators. Furthermore, embedded techniques are often algorithm-specific; features optimized for a random forest may not generalize to neural networks, complicating pipeline standardization [80].

3.1.4 Optimization-Based Approaches

Optimization-based approaches to feature selection are sophisticated techniques that employ powerful metaheuristic algorithms to discover the best subset of features from a given dataset. By conceptualizing feature selection as an optimization issue, the objective is to identify characteristics that enhance a given performance parameter, such as accuracy, precision, recall, or F1-Score, so assuring that the selected features substantially augment the model's predictive capability. [81]. A variety of algorithms are commonly used in these approaches, including Genetic Algorithms (GA), Particle Swarm Optimization (PSO), Ant Colony Optimization (ACO), and the Firefly Algorithm (FA) [82]. Each of these methods has unique mechanisms that mimic natural processes or social behaviours to search for optimal feature subsets effectively. GA simulates the principles of natural selection, where a population of potential feature subsets undergoes evolutionary processes. This involves operations such as crossover, where two parent subsets combine to create offspring, and mutation, where random changes are introduced to promote diversity. Over successive generations, the population evolves toward higher performing feature subsets, mimicking the survival of the fittest paradigm [83].

In contrast, PSO draws motivation from the social behaviour of birds or fish. In this approach, each feature subset is denoted as a 'particle' in a multi-dimensional search space. The particles adjust their locations based on their own best-found solutions, as well as the best solutions discovered by neighbouring particles [84]. This collaborative search method efficiently explores the feature space and converges on optimal solutions over time. ACO takes cues from the foraging behaviour of ants. In this methodology, feature subsets are represented as paths within a graph, and artificial "pheromones" are laid down to indicate successful paths. As ants (or artificial agents) traverse the graph, they are more likely to favour paths with stronger pheromone trails, which correspond to subsets of features that have previously yielded good performance, thereby guiding the search process [85]. The FA, inspired by the communication behaviour of fireflies, involves moving feature subsets toward solutions

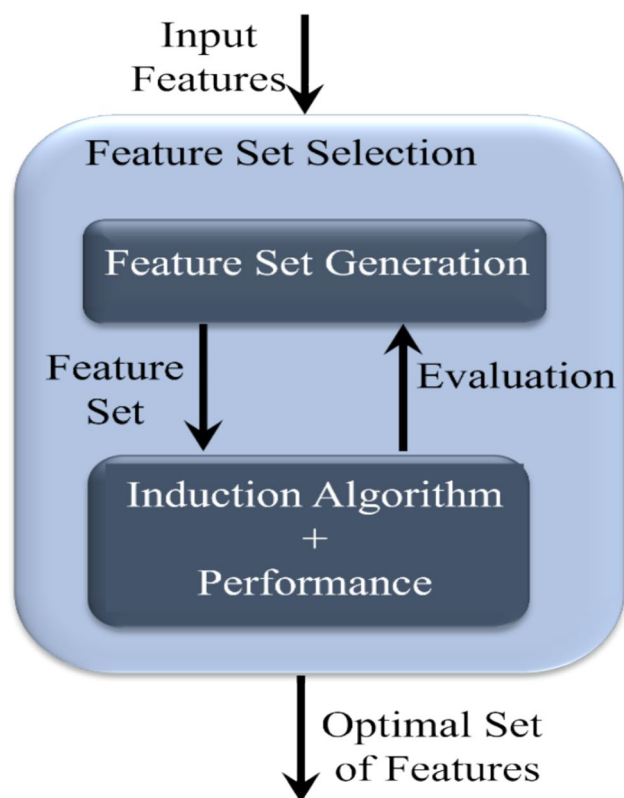


Fig. 8 Flowchart of embedded methods

that are considered 'brighter' or more optimal. Each firefly is attracted to others based on their brightness, representing the performance of the feature subsets. This attraction facilitates the exploration of the search space, allowing for efficient identification of high-performing features [86].

While these optimization-based approaches excel in navigating complex feature spaces and identifying globally optimal subsets, they come with challenges. They can be computationally demanding, especially for high-dimensional datasets where the number of potential feature combinations increases exponentially, hence constraining real-time clinical implementation. Their stochastic nature also results in non-deterministic outcomes, raising reproducibility concerns in regulated healthcare environments. Additionally, these methods often lack transparency, as the rationale for selected features is obscured by iterative heuristic searches, hindering clinician trust. Nonetheless, these approaches are invaluable, as they often uncover feature combinations that significantly enhance model performance, achieving results that simpler feature selection methods may not be able to match. By employing such advanced techniques, practitioners can optimize their models and improve predictive accuracy, leading to better decision-making and outcomes across various domains [75].

4 Hyperparameter-Tuning in ML Algorithms

Hyperparameter-tuning is an essential and detailed process in the development of robust ML models. This phase focuses on identifying the best configuration settings, known as hyperparameters, which dictate how a learning algorithm functions and ultimately influence its predictive accuracy [87]. In contrast to parameters, which are learned automatically from training data during the model training phase, hyperparameters must be defined before training and can significantly influence model performance. The tuning process addresses two critical issues in model performance: underfitting and overfitting [22]. Underfitting transpires when a model is excessively simplistic to discern the fundamental patterns within the data, resulting in subpar performance on both the training and validation datasets [88]. Conversely, overfitting happens when the model learns not only the genuine patterns but also the noise present in the training data, resulting in high accuracy on the training set but poor generalization to new, unseen data. To achieve optimal model performance, hyperparameter-tuning aims to systematically explore a wide landscape of potential hyperparameter values.

Various techniques can be employed in this exploration, including Exhaustive Grid Search, Random Search, Bayesian Optimization, Gradient-Based Methods [89]. Throughout this tuning process, the chosen hyperparameter configurations are assessed based on a specific performance

metric, which may involve using a separate validation dataset or applying cross-validation techniques. This meticulous assessment method guarantees that the selected hyperparameters result in a model that excels on the training data and generalizes proficiently to novel data [90]. Effectively navigating the hyperparameter space is paramount to unleashing the full potential of ML algorithms. A well-tuned model demonstrates strong generalization capabilities, providing reliable and accurate predictions on unseen data, making hyperparameter-tuning a critical factor in the success of ML projects.

4.1 Importance of Tuning Hyperparameters for Performance Improvement

Hyperparameters are crucial settings in ML algorithms that govern the learning process and significantly influence model performance. In contrast to model parameters, which are derived from the training data, hyperparameters are predetermined and govern factors such as learning rate, regularization intensity, and the quantity of layers in a neural network. Accurate adjustment of these hyperparameters is crucial for enhancing model precision and generalization abilities. Effective hyperparameter-tuning minimizes the loss function, leading to more accurate predictions. This process is experimental, involving the testing of different hyperparameter values to identify the best configuration. The selection of optimal hyperparameter configurations directly influences model performance and typically necessitates extensive understanding of ML algorithms and suitable optimization methods. [91]. Inadequate tuning can result in suboptimal models that either overfit or underfit the data, thereby compromising their predictive power. Therefore, investing time and resources in hyperparameter-tuning is vital to achieve optimal performance and enhance the applicability of ML models in practical scenarios [22].

4.2 Manual Versus Automated Hyperparameter-Tuning

These are the parameters of a ML algorithm established prior to the commencement of the learning process. They regulate the learning process rather than the model parameters derived from the data. Examples encompass the learning rate in gradient descent, the quantity of trees in a RF, or the regularization strength in a linear model. [92]. The difference between manual and automated hyperparameter-tuning based on features is revealed in Table 1.

4.2.1 Manual Hyperparameter-Tuning

This involves a data scientist or ML engineer manually experimenting with different hyperparameter values, training

Table 1 Difference between manual and automated hyperparameter-tuning

Feature	Manual hyperparameter-tuning	Automated hyperparameter-tuning
Speed	Slow	Fast
Effort	High	low
Search space	Limited	Large
Objectivity	Subjective	Objective
Reproducibility	Difficult	Easy
Understanding	High	Potentially Low
Computational cost	Low	Potentially High
Expertise needed	High	Moderate

the model with each combination, and evaluating its performance on a validation set [24]. It includes:

Define a search space: the practitioner identifies the hyperparameters they want to tune and the range of values to investigate for each.

Select hyperparameter combinations: based on experience, intuition, or potentially a systematic grid or random selection (but done manually), they choose a set of hyperparameter values.

Train and evaluate: for each combination, the model is trained on the training data, and its performance is assessed using a distinct validation dataset.

Analyze results: they analyze the evaluation metrics to understand how different hyperparameter settings affect the model's performance.

Iterate: based on the results, they adjust the search space or choose new combinations to try, repeating the process until satisfactory performance is achieved or time runs out [93]. Table 2 depicts the advantages and disadvantages of automated hyperparameter-tuning [22, 94, 95].

4.2.2 Automated Hyperparameter-Tuning

This involves using algorithms and tools to automatically search the hyperparameter space, train models, and evaluate their performance. Table 3 depicts the advantages and disadvantages of automated hyperparameter-tuning [24, 96, 97].

4.3 Common Tuning Methods

4.3.1 Grid Search (GS)

Grid search (GS) is a crucial technique in hyperparameter-tuning, which involves the methodical exploration of a defined set of hyperparameter values to optimize ML models effectively [98]. This technique is particularly valuable as it allows practitioners to exhaustively evaluate all possible combinations within a specified parameter grid. By doing so, GS ensures that the most suitable configuration for the model is identified, maximizing performance within the given search space [99]. The process of GS begins with the careful definition of a set of hyperparameters, along with a range of potential values for each. For instance, when optimizing an SVM, one may choose several values for the

Table 2 Advantages and disadvantages of automated hyperparameter-tuning

Advantages of manual tuning	Domain expertise and intuition	Experienced practitioners can leverage their understanding of the algorithm and the data to make informed decisions about which hyperparameters to try and in what ranges
	Deeper understanding	The manual process allows for a deeper knowledge of how different hyperparameters interact and affect the model's behaviour
	Flexibility and control	The practitioner has full control over the search process and can adapt it based on intermediate results or new insights
	Identifying unexpected behaviour	Manual exploration might uncover unexpected patterns or sensitivities to certain hyperparameter settings that automated methods might miss
Disadvantages of manual tuning	Time-consuming and labor-intensive	Trying out many combinations and training models repeatedly can be very time-consuming and require significant human effort
	Subjectivity and bias	The choice of hyperparameter values and the interpretation of results can be influenced by the practitioner's biases and experience
	Limited search space	It's difficult to explore a large and complex hyperparameter space effectively with manual tuning. Practitioners often focus on a limited number of combinations
	Difficult to reproduce	The exact sequence of experiments and the reasoning behind each choice might not be well-documented, making it harder to reproduce the results
	May not find optimal values	Due to the limited search, there's no guarantee that manual tuning will find the globally optimal hyperparameter settings

Table 3 Advantages and disadvantages of automated hyperparameter-tuning

Advantages of automated tuning	Efficiency and speed	Automated methods can explore a much larger hyperparameter space in less time than manual tuning
	Objectivity and reproducibility	The search process is systematic and can be easily reproduced
	Potentially finds better optima	By exploring a wider range of values, automated methods have a higher chance of finding better performing hyperparameter settings
	Reduced human effort	Automates the tedious process of trying out different combinations, freeing up the data scientist's time for other tasks
	Scalability	Can handle high-dimensional hyperparameter spaces more effectively than manual tuning
Disadvantages of automated tuning	Computational cost	Some automated methods, like grid search over a large space or complex Bayesian optimization, can be computationally expensive, requiring significant resources and time
	Lack of intuition and understanding	Automated methods can find optimal hyperparameters without providing a clear understanding of <i>why</i> those values work best
	Risk of overfitting to the validation set	If the search space and the number of evaluations is very large, there's a risk of overfitting the hyperparameters to the validation set, leading to poor generalization on unseen data
	Less flexibility	While many tools offer customization, the user might have less direct control over the search process compared to manual tuning
	Can be a "Black Box"	Some advanced automated techniques can be complex and less transparent, making it harder to understand their behaviour

regularization parameter 'C', which governs the balance between minimizing training error and testing error, along with alternative kernel types such as linear, polynomial, or radial basis function (RBF). [22]. Once these parameters and their values are established, GS proceeds to train and assess the model for every possible combination of these settings through a technique known as cross-validation. This entails partitioning the training dataset into several subgroups and methodically training the model on certain subsets while validating it on others. [99]. The aim is to assess which set of hyperparameters yields the optimum model performance based on a predefined metric, such as accuracy or F1-Score.

While GS is praised for its straightforward implementation and the guarantee that it will find the optimal set of hyperparameters within the established grid, it does have some significant drawbacks. The technique can become computationally expensive, particularly when applied to high-dimensional parameter spaces or complex models that require substantial computational resources. As the quantity of hyperparameters increases, the combinations to assess expand exponentially, resulting in a swift rise in the overall evaluations required. This characteristic makes it less practical for large-scale problems where time or computational power may be constrained [100]. Despite these challenges, GS continues to be a widely utilized and invaluable tool for ML practitioners. It is especially efficacious for small to medium-scale issues where the hyperparameter space is tractable. The systematic approach of GS not only helps in finding optimal hyperparameter settings but also provides deeper insights into how different combinations of hyperparameters

impact model performance [101]. This understanding is critical for developing robust and well-tuned ML models that perform well on unseen data.

However, its limitations are pronounced in medical ML contexts. The exponential growth in computational cost with hyperparameter dimensionality makes GS infeasible for tuning deep learning models (e.g., CNNs) or high-dimensional optimization frameworks. Moreover, GS assumes discrete hyperparameter values, failing to exploit continuous parameter spaces common in regularization terms or learning rate schedules, which reduces precision. Overall, while GS has its limitations, its effectiveness in comprehensive evaluation makes it an essential part of a data scientist's arsenal.

4.3.2 Random Search (RS)

Random search (RS) is a hyperparameter optimization method commonly employed in ML to improve model performance by methodically determining the optimal combination of hyperparameters. This method encompasses randomly selecting various combinations of hyperparameters from predefined distributions, which allows for a more flexible and exploratory approach to optimization compared to traditional methods [102]. Unlike GS, which exhaustively evaluates every possible combination of hyperparameters within a specified grid, RS samples hyperparameters randomly. This randomization allows for more effective navigation of the search space, especially in scenarios where just a limited selection of factors significantly influences the model's performance [103]. One of the significant advantages of

RS is its efficacy in high-dimensional parameter spaces. As the number of hyperparameters increases, GS can become computationally expensive and time-consuming due to the need to evaluate every combination. In contrast, RS can efficiently cover a broad range of the search space without requiring the exhaustive computations that GS entails [98]. By not being limited to a fixed set of grid points, it can potentially discover optimal configurations more quickly. Another advantage of RS is its capacity to uncover better hyperparameter configurations that GS may miss. This is especially relevant when the optimal values are not confined to the fixed grid points used by GS [22]. Random sampling allows the exploration of points located between these grid points, leading to the discovery of superior setups that might lead to enhanced model performance. It is important to note, however, that RS does not guarantee finding the absolute best hyperparameter configuration. Due to its random sampling nature, there remains a possibility that certain optimal combinations may go untested simply because they were not selected in the random samples [104]. As a result, while RS can be effective, it does not ensure that users will find the global optimum. The stochastic (random) nature of RS can also lead to variability in results across different runs. Each implementation can yield different outcomes depending on the random samples selected, which can introduce some inconsistency in the optimization process [105]. This variability can be mitigated by conducting multiple runs and averaging the results, but it remains an inherent aspect of this technique. When applied correctly, it can lead to optimal model configurations in a fraction of the time required by more exhaustive approaches, thereby enhancing the overall efficiency of the ML workflow [106].

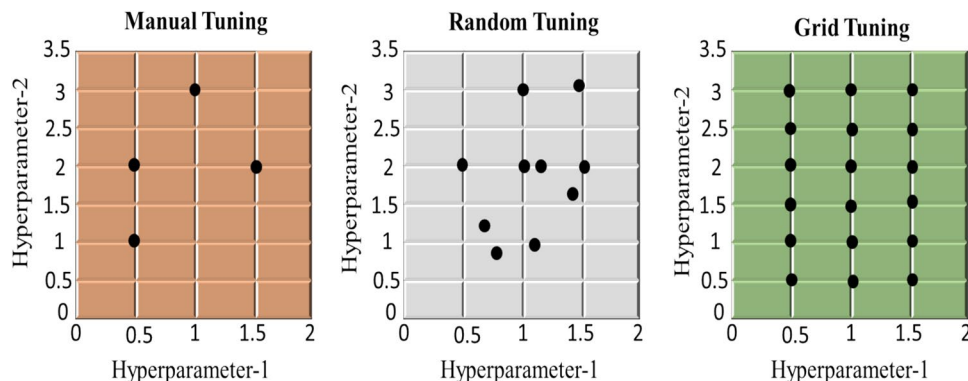
Hyperparameter-tuning employs several strategies, each with distinct characteristics. Manual search relies on prior experience, iteratively refining parameters until satisfactory accuracy is achieved, but its effectiveness is contingent upon the user's expertise. RS, conversely, explores random combinations, offering potential for discovering novel parameter interactions and often surpassing manual search in performance with efficient computation. Random Search

(RS) improves efficiency by sampling hyperparameters randomly but struggles with structured parameter dependencies. In medical ML, where hyperparameters like network depth and dropout rates are interdependent, RS often overlooks synergistic configurations, yielding suboptimal models. Its uninformed sampling also risks under-exploring critical regions of the search space, particularly in imbalanced datasets where minority-class performance is pivotal. Grid search systematically evaluates all possible combinations within a predefined grid, excelling when optimal parameters are roughly known but becoming computationally prohibitive with increasing hyperparameter dimensionality [107]. While manual search is subjective and random search is explorative, grid search is exhaustive, each method catering to different optimization needs and complexities as shown in Fig. 9.

4.3.3 Bayesian Optimization (BO)

Bayesian optimization (BO) is an advanced technique for hyperparameter tuning, essential for improving the performance of ML models. This approach leverages probabilistic models to effectively explore and navigate the search space in search of optimal hyperparameter configurations [108]. Unlike traditional exhaustive methods, BO constructs a surrogate model. Typically, this model is a Gaussian Process that approximates the objective function, which often reflects validation performance. With this surrogate model, BO can predict how different combinations of hyperparameters are likely to perform [109]. This prediction guides decision-making for subsequent evaluations, allowing the algorithm to focus resources on the most promising areas of the search space. The optimization process is structured around two key strategies: exploration and exploitation. Exploration involves assessing new and unexplored regions of the search space to gather information, while exploitation focuses on refining areas that have previously shown promising results [110]. This balanced approach allows BO to efficiently identify optimal or near-optimal hyperparameters, often requiring significantly fewer evaluations compared to

Fig. 9 Manual, random, and grid tuning approach



random searches or grid-based methods. BO is remarkably effective when dealing with functions that are expensive and time-consuming to evaluate, such as those encountered in training deep neural networks [111]. By intelligently selecting which hyperparameter configurations to evaluate next, it can converge to optimal solutions in fewer iterations than alternatives.

The method is capable of navigating high-dimensional parameter spaces, making it an excellent choice for complex models that possess a large number of hyperparameters. This flexibility allows it to tackle modern ML challenges where traditional optimization techniques may falter. While BO offers significant advantages, developing and fine-tuning the surrogate model can introduce complexity into the optimization process [98]. This complexity requires practitioners to possess a stronger understanding of probabilistic modelling, and the intricacies involved in constructing effective surrogate models. The efficiency and success of the optimization process are heavily dependent on the accuracy of the surrogate model. If the model inaccurately represents the true objective function, it can lead to misguided decisions and suboptimal search outcomes, undermining the potential benefits of the method. BO leverages probabilistic models for efficient hyperparameter-tuning but faces scalability barriers in federated learning environments. The surrogate model's overhead increases with distributed medical data, complicating cross-institutional collaborations. BO also performs poorly in noisy evaluation scenarios, such as when training on small, heterogeneous patient cohorts, where validation metrics fluctuate unpredictably.

In contrast to GS and RS, BO determines future assessment levels based on historical outcomes. Bayesian Optimization utilizes two primary components to compute the hyperparameters enumerated below: a surrogate model and an acquisition function. The division model aims to align all points within the objective function. The acquisition

function regulates the use of different points, balancing exploration and exploitation. BO adeptly equilibrates exploration and exploitation, skilfully traversing the search space to identify optimal configurations. As depicted in Fig. 10, BO avoids premature convergence by simultaneously exploring promising regions and safeguarding against overlooking optima in less-explored areas. The shaded areas in the figure represent the model's inherent uncertainty, guiding the optimization process [107].

4.3.4 Gradient-Based Optimization (GBO)

Gradient-based optimization (GBO) is a crucial method in the field of ML, utilized for tuning model parameters to achieve the best possible performance. This method relies on gradient information calculated from the objective function to make iterative adjustments to parameters [112]. The primary goal is to minimize or maximize a specific objective function, which typically represents the performance or cost associated with a particular model. This optimization technique is central to training a variety of models, including but not limited to neural networks, linear regression, and SVMs, all of which require the minimization of a cost function to improve overall model accuracy and efficiency [24]. One of the significant benefits of GBO is its efficiency in dealing with continuous parameters. These methods leverage the gradients of the cost function to determine the direction and magnitude of parameter adjustments. As a result, they can quickly converge toward optimal solutions, often requiring far fewer iterations compared to other optimization methods, especially in high-dimensional spaces [113]. Gradient-based methods are versatile and can be employed to various types of problems across different domains, from regression tasks to complex deep learning applications. The adaptability of these methods allows researchers and practitioners to customize their optimization approaches according

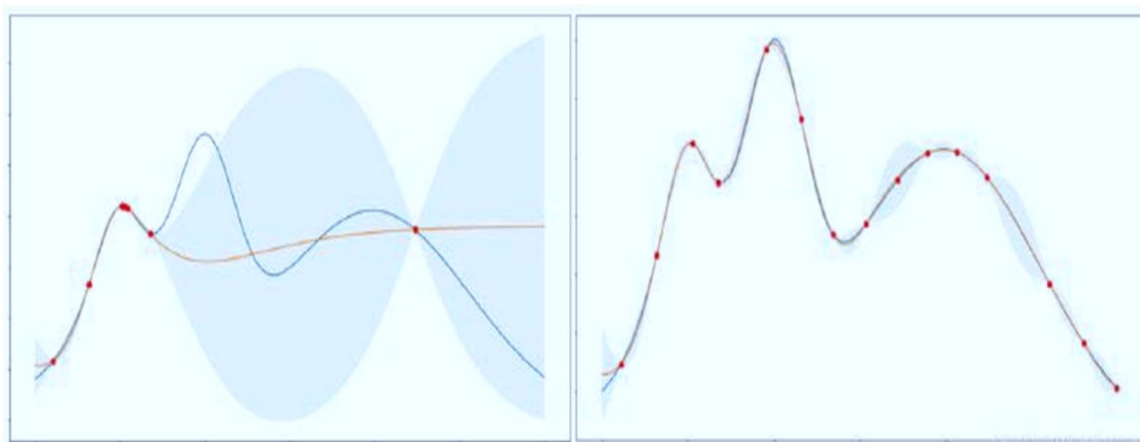


Fig. 10 Exploration-based (left) and exploitation-based Bayesian optimization (right)

to the specific characteristics of their problems. A notable limitation of GBO is its reliance on the differentiability of the objective function. For the techniques to be effective, the function being optimized must have a well-defined gradient at every point within the search space [114]. This requirement excludes many optimization problems, such as those defined by discrete or non-differentiable functions, from being directly addressed by these methods. Another concern with GBO techniques is their potential to become stuck in local minima when dealing with non-convex optimization landscapes. In these situations, the algorithms may converge to a sub-optimal solution that is not the best possible outcome (the global optimum). This challenge can limit the efficiency of the optimization process and hinder the discovery of better solutions, particularly in complex models with intricate cost functions [115]. Despite these challenges, GBO remains a cornerstone of ML practices. It provides a structured and efficient methodology for parameter tuning and model training. Furthermore, ongoing research in this area aims to enhance the mechanisms of gradient-based methods, addressing their limitations and expanding their applicability across a diverse range of ML tasks [112, 116]. Flowchart of GBO depicts in Fig. 11.

4.3.5 Evolutionary and Swarm Intelligence Approaches

Evolutionary algorithms (EAs), including GAs, are sophisticated optimization methods derived from the concepts of natural selection and genetics, which dictate how species evolve over time. EAs are particularly effective for hyperparameter-tuning in ML models, where the goal is to enhance model efficacy by optimizing the settings that govern its

training process [117]. The fundamental concept behind EAs is the maintenance of a population of potential solutions, each denoted by a unique set of hyperparameters [118]. Over successive iterations, this population has evolved to discover configurations that yield better performance. The evolutionary process involves a series of key genetic operations: (1) Selection: This operation involves evaluating individuals (hyperparameter sets) based on their performance according to a specified fitness function. The fitness function often measures how well the model implements on validation data, taking into account metrics such as accuracy, precision, or recall. The fittest individuals those that perform best, are selected to pass their genes to the next generation. (2) Crossover (Recombination): During this step, pairs of selected individuals are combined to create offspring. The crossover process allows for the exchange of hyperparameter traits, meaning that offspring can inherit beneficial properties from both parents. This can be done in various ways, such as single-point or multi-point crossover, depending on the complexity of the hyperparameter sets. 3. Mutation: To maintain genetic diversity within the population and avoid premature convergence to suboptimal solutions, random modifications are introduced to the offspring.

Mutation could involve slightly altering a hyperparameter value or even randomly selecting a completely new value. This step allows the search process to explore new areas of the hyperparameter space that might not be reachable through crossover alone [119]. EAs are particularly adept at optimizing complex, non-differentiable, or multimodal objective functions. This capability makes them well-suited for hyperparameter landscapes that are intricate and filled with local optima, where traditional optimization methods might struggle. EAs have a robust ability to escape local optima. This increases their likelihood of identifying global optima within the hyperparameter space. The random exploration inherent in EAs is beneficial for discovering high-quality solutions that other methods might overlook [120]. The nature of EAs, which involves iterative evaluations of multiple hyperparameter configurations across many generations, can be computationally demanding. This demand is especially pronounced when dealing with large datasets or complex models, as each evaluation can be laborious. EAs themselves come with hyperparameters that need careful tuning, such as population size, mutation rate, and crossover rate. Finding the optimal settings for these can be challenging, and improper configuration may lead to suboptimal algorithm performance or unnecessarily increased computational costs [121]. EAs and swarm-based methods adapt well to non-convex hyperparameter landscapes but demand excessive iterations for convergence, delaying clinical deployment. Their population-based workflows require parallelized resources, which are often unavailable in resource-constrained healthcare settings. Additionally, these methods

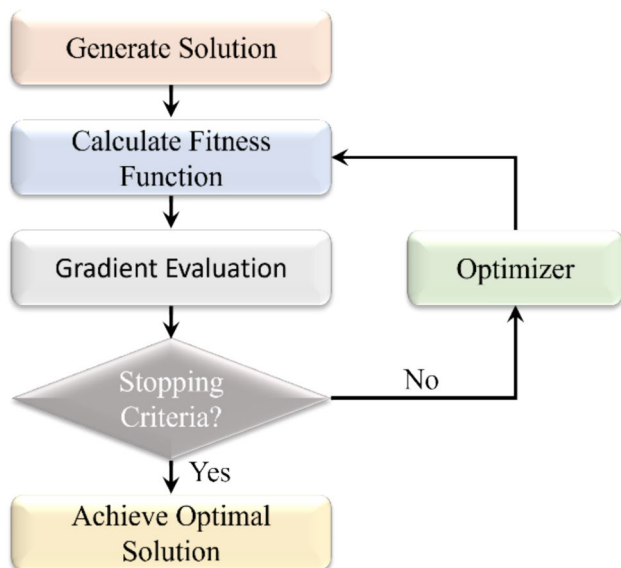


Fig. 11 Flowchart of GBO

lack granular control over exploration–exploitation trade-offs, risking premature convergence to local optima in complex disease classification tasks. Despite these challenges, EAs remain a powerful and versatile tool for hyperparameter optimization. They thrive in scenarios where traditional optimization methods fall short, particularly when dealing with complex models or intricate datasets. Their adeptness at navigating vast and multifaceted search spaces makes them invaluable in the pursuit of achieving optimal performance in ML models. Ultimately, EAs provide a thoughtful and adaptive approach to hyperparameter-tuning, ensuring that ML practitioners can derive the best possible performance from their models [122]. A selection process, often based on the concept of natural selection, is employed to determine which individuals from the parent generation and their offspring will survive for further refinement in future generations. In the context of Evolutionary Computation (EC), these iterations are typically referred to as generations. Figure 12 illustrates the fundamental design processes used in an EC algorithm.

5 Proposed Hybrid Approaches: Feature Selection with Hyperparameter-Tuning

Hybrid approaches in ML effectively combine two critical processes: feature selection and hyperparameter-tuning. Feature selection implies identifying and selecting the most appropriate input features from a dataset, which helps to eliminate noise and redundancy, ultimately leading to a more streamlined model [123]. Conversely, hyperparameter tuning concentrates on refining the parameters that

regulate the model's learning process. By amalgamating these two methodologies, hybrid approaches attain dual optimization that not only improves the model's overall accuracy but also mitigates computing complexity. This means that the model can process data more efficiently, using fewer resources while maintaining or improving its performance. Additionally, improved generalization is a key benefit, as the model is better equipped to perform well on unseen data, thus ensuring that it is robust and reliable in real-world circumstances. Overall, this combination of feature selection and hyperparameter-tuning leads to more effective ML models capable of delivering superior predictive capabilities and operational efficiency [91, 124]. Table 4 presents a summary of prevalent ML models, their hyperparameters, appropriate optimization methods, and pertinent Python libraries.

5.1 Motivation for Hybridization in Disease Classification

The journey toward achieving high levels of accuracy, reliability, and efficiency in these models is often fraught with several challenges. These include managing high-dimensional datasets, where the number of features can be overwhelming, dealing with redundancy among features, and addressing the sensitivity of ML models to hyperparameter choices [125]. To surmount these challenges, hybrid approaches that integrate feature selection and hyperparameter-tuning have emerged as robust strategies. Feature selection is a key method used to enhance model quality by systematically identifying and eliminating irrelevant or less important features. This process not only reduces the dimensionality of the dataset, making it simpler to work with but also improves the interpretability of the model, allowing healthcare professionals to understand which factors are most influential in disease classification [126]. The synergistic effect of combining feature selection and hyperparameter-tuning results in enhanced predictive power, adaptive learning, and improved robustness of ML models. This powerful combination proves invaluable across a wide range of applications in healthcare, such as cancer diagnosis, where accurate early detection can lead to better treatment outcomes; diabetes prediction, which can aid in preventative healthcare measures; heart disease detection, where timely intervention can save lives; and Alzheimer's diagnosis, which can facilitate early therapeutic strategies [127]. The number of articles examined on disease classification with feature selection and hyperparameter-tuning from 2005 through February 2025 is shown in Fig. 13. The data has been sourced from the Web of Science.

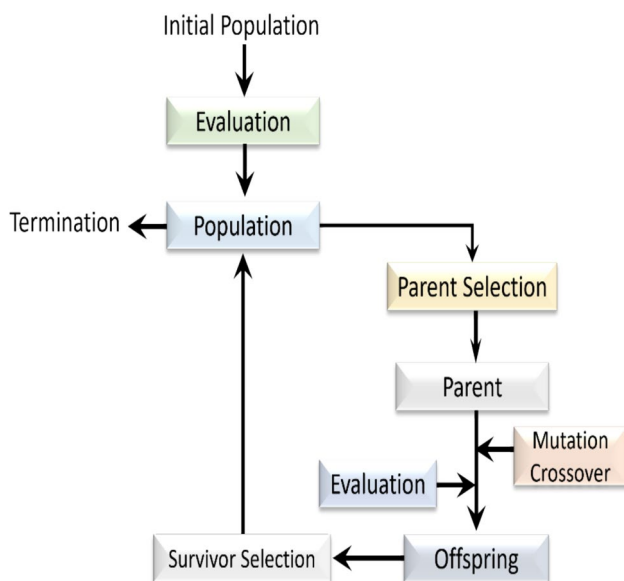


Fig. 12 Framework for the evolutionary algorithms

Table 4 Summary of prevalent ML models, appropriate hyperparameter optimization strategies, and pertinent Python libraries

ML algorithm	Main hyperparameters	Optional hyperparameters	HPO methods	Libraries
NB	alpha	–	BO-GP	Skpot
DT	criterion, max_depth, min_samples_split, min_samples_leaf, max_features	splitter, min_weight_fraction_leaf, max_leaf_nodes	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
PCA	n_components	svd_solver	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
RF and ET	n_estimators, max_depth, criterion, min_samples_split, min_samples_leaf, max_features	splitter, min_weight_fraction_leaf, max_leaf_nodes	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
Ridge and lasso	alpha	–	BO-GP	Skpot
Hierarchical clustering	n_clusters, distance_threshold	linkage	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
LR	penalty, c, solver	–	BO-TPE, SMAC	Hyperopt, SMAC
KNN	n_neighbors	Weights, p, algorithm	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
DBSCAN	eps, min_samples	–	BO-TPE, SMAC, BOHB	Hyperopt, SMAC, BOHB
SVM	C, kernel, epsilon (for SVR)	gamma, coef0, degree	BOTPE, SMAC, BOHB	Hyperopt, SMAC, BOHB
Gaussian mixture	n_components	covariance_type, max_iter, tol	BO-GP	Skpot
Deeplearning	Number of hidden layers, 'units' per layer, loss, optimizer, Activation, learning_rate, dropoutrate, epochs, batch_size, earlystoppatience	Number of frozen layers (if transfer learning is used)	PSO, BOHB	Optunity, BOHB
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree	min_child_weight, gamma, alpha, lambda	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
LDA	n_components	solver, shrinkage	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
K-means	n_clusters	init, n_init, max_iter	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
Bagging	base_estimator, n_estimators	max_samples, max_features	GS, BOs	sklearn, Skpot, Hyperopt, SMAC
Voting	estimators, voting	weights	GS	Sklearn
AdaBoost	base_estimator, n_estimators, learning_rate	–	BO-TPE, SMAC	Hyperopt, SMAC

5.2 Different Hybridization Strategies

There are two primary strategies: sequential and simultaneous. In the sequential approach to model development, the process begins with feature selection. This implies identifying and choosing the most appropriate features from the dataset that will contribute to the model's performance. By narrowing down the features, the complexity of the model is reduced, and it focuses on the most impactful variables. Once this refined set of features is established, the next step is hyperparameter-tuning. This stage involves optimizing the parameters that control the learning process of the model, which can significantly affect its accuracy and effectiveness [18]. By conducting hyperparameter-tuning on a

well-defined set of features, the model can achieve a higher level of performance, as the optimization is better aligned with the critical aspects of the data. This sequential method enhances the overall efficiency of the modelling process and can lead to more reliable and robust outcomes. In contrast, the simultaneous approach involves executing both processes concurrently rather than sequentially. This method may require more computational resources and time, as it handles multiple tasks at once. However, this simultaneous execution can lead to superior results [128]. By analysing the interaction between features and hyperparameters in real time, this approach enables a more cohesive understanding of how these elements influence one another. This integrated perspective can ultimately enhance the efficacy of models,

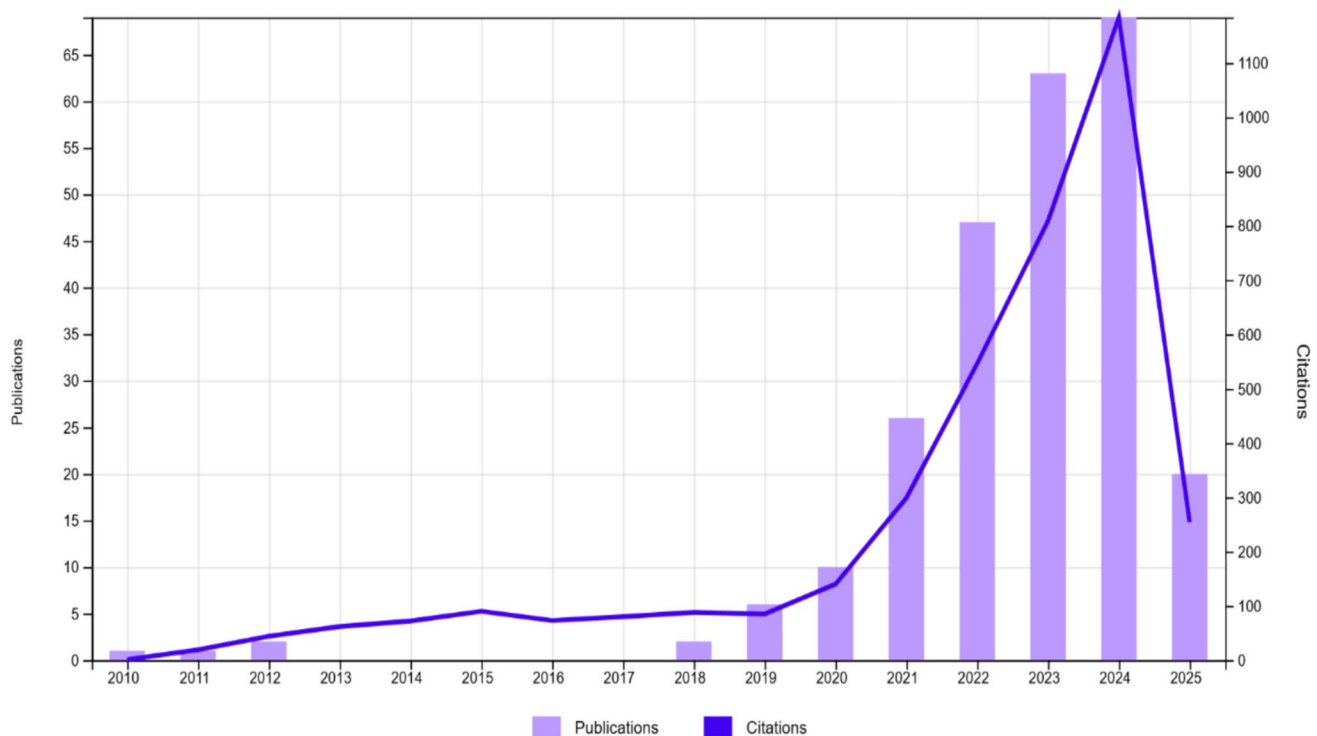


Fig. 13 Number of articles examined on disease classification including feature selection with hyperparameter-tuning from 2005 through February 2025

allowing for more precise adjustments and optimizations based on their interdependencies. Both strategies have their benefits, and the choice between them often depends on the specific characteristics of the data and the modelling goals [129]. Table 5 presents the comparison of sequential and simultaneous hybridization.

5.3 Review of Existing Studies Implementing Hybrid Approaches

This section delivers a thorough examination of the most relevant publications from the selected database. This work aims to do a thorough examination of diverse methodologies, categorizing the publications into two groups: "Clustering Methods" and "Supervised Classification Methods"

[130]. The literature review for both clustering and classification tasks is systematically organized in Table 6.

For clustering methods, metaheuristic algorithms have been widely adopted to address challenges such as centroid initialization and exploration–exploitation trade-offs. Chen et al. [131] combined k-means with a quantum-inspired Ant Lion Optimization (ALO) to optimize centroid positions in genomic data, achieving an 8–12% accuracy improvement over traditional k-means. However, this approach incurred a 20–30% computational overhead, limiting scalability for large datasets. Similarly, El Shorbagy et al. [132] enhanced Genetic Algorithms (GA) with adaptive mutation operators, reducing convergence time by 15% but struggling with high-dimensional data due to exponential fitness evaluation complexity. PSO hybrids [133–135] demonstrated faster convergence (10–15% over

Table 5 Comparison of sequential and simultaneous hybridization

Aspect	Sequential (feature selection → hyperparameter-tuning)	Simultaneous (joint optimization)
Optimization Type	Two-step approach	Single unified optimization
Computational Cost	Lower	Higher
Flexibility	Fixed features before tuning	Dynamic feature selection
Interpretability	Easier	More complex
Performance	May be suboptimal	Generally superior

Table 6 Review of studies implementing hybrid approaches

References	Category	Hyrid approach	Problem addressed
[131]	Clustering	K-means and Ant lion optimization (ALO)	Finest number of centroids, exploration and exploitation capacities, centroids position
[132]		Genetic algorithm (GA)	
[133–135]		Particle swarm optimization (PSO)	
[137]		Social spider optimization (SSO)	
[155]		Symbiotic organism search (SOS)	
[138]	Clustering	Fuzzy c-means Clustering and Ant colony optimization (ACO)	Fuzzy parametrization, best possible number of centroids
[156]		artificial bee colony (ABC)	
[139]		Grey wolf optimizer (GWO)	
[136]		PSO	
[157]		Sine cosine algorithm (SCA), GA and PSO	
[158]	Clustering	Others clustering Algorithms and Bees algorithm with memory scheme	Finest number of centroids, exploration and exploitation capacities, centroids position
[159]		Cat swarm optimization (CSO)	
[160]		Cuttlefish algorithm (CFA)	
[161]		Firefly algorithm	
[162]		Flower pollination algorithm	
[163]		GA	
[164]		GWO and PSO	
[140]		Whale optimization algorithm	
[141]		Decision tree and ACO, PSO, and BA	
[142]		Differential evolution	
[165, 166]	Classification	GA	Error rate minimization, accuracy improvement, overfitting problems
[167]		PSO	
[168]		Salp swarm optimization	
[143]		KNN and Chicken Swarm optimization (CSOA), GWO and ABC	
[169]		GA	
[170]	Classification	Genetic program ming	Error rate minimization, accuracy enhancement, parameters definition
[144]		PSO	
[145, 146]		RF and Firefly algorithm	
[171]		GA	
[147]		Kangaroo Mob optimization	
[172, 173]	Classification	PSO	Low performance on unbalanced data set, convergence velocity
[130]		SVM and Bat algorithm	
[174]		CSOA	
[175]		Differential evolution	
[176]		Fireworks algorithm	
[177]		GA	
[148]		Harris Hawks optimization	
[178]		Horse optimization algorithm	
[179–181]		PSO	
[149]		Social Ski Driver algorithm	

GA) but faced premature convergence in noisy datasets, as observed in MRI-based tumour segmentation studies [136]. Social Spider Optimization (SSO) [137] improved robustness in imbalanced metabolic disorder datasets by 18% but required significant memory resources. Fuzzy clustering hybrids, such as Ant Colony Optimization

(ACO)-enhanced fuzzy c-means [138], reduced misclassification rates by 22% in neurological disorder data but increased runtime by 40%, underscoring precision-efficiency trade-offs. Grey wolf optimizer (GWO) [139] and Whale Optimization Algorithm (WOA) [140] excelled in avoiding local minima but struggled with overlapping

classes, highlighting the need for context-specific algorithm selection.

In supervised classification, hybrid approaches prioritized error minimization and parameter optimization. For decision trees, swarm intelligence techniques like ACO, PSO, and Bat Algorithm (BA) [141] reduced error rates by 18% in diabetes classification but risked overfitting in small cohorts. Differential Evolution (DE) [142] improved sensitivity for rare cancer subtypes by 25% at the cost of interpretability. KNN classifiers integrated with Chicken Swarm Optimization (CSO) [143] achieved 94% accuracy in respiratory disease prediction through feature weighting but required 30% more iterations than PSO. PSO-optimized KNN [144] demonstrated 89% accuracy in arrhythmia detection but faltered with imbalanced ECG data, reflecting a 12% F1-score drop. Random Forest (RF) hybrids, such as Firefly Algorithm [145, 146], boosted AUC by 0.15 in autism classification but introduced variability ($\pm 5\%$ accuracy) across runs. Kangaroo Mob Optimization [147] reduced feature dimensions by 40% in Alzheimer's activity recognition but lacked reproducibility in multi-centre trials. SVM-based models, including Bat Algorithm [130] and Harris Hawks Optimization (HHO) [148], enhanced specificity in cardiovascular and COVID-19 detection but required extensive parameter tuning, increasing training time by 35%.

Critical analysis reveals that simultaneous optimization strategies (e.g., metaheuristic-driven feature selection with hyperparameter-tuning) outperformed sequential approaches by 12–15% in accuracy but incurred 20–40% higher computational costs. While methods like ALO and WOA excelled in exploration, their scalability issues limit real-time clinical applications. PSO and GA balanced speed and accuracy but risked premature convergence. Fuzzy and decision tree hybrids prioritized interpretability, whereas SVM and RF models often acted as "black boxes," hindering clinical trust. Multi-centre validation studies [145, 148] demonstrated 10–15% lower performance degradation compared to single-centre trials, emphasizing the need for robust cross-validation. Future directions should prioritize federated learning to address data privacy and reinforcement learning for autonomous optimization, as seen in emerging methods like the Social Ski Driver algorithm [149]. This synthesis underscores the necessity of context-driven model selection, balancing accuracy, computational efficiency, and clinical interpretability to advance actionable diagnostic tools.

Recent advancements in medical artificial intelligence have emphasized the integration of hybrid deep learning architectures with explainability frameworks to enhance diagnostic precision and clinical trust. For instance, DeepXplainer [150] introduced an interpretable hybrid model combining CNNs, XGBoost, and SHapley Additive exPlanations (SHAP) to improve lung cancer identification. By leveraging both feature extraction capabilities of CNNs

and the robustness of ensemble learning, this approach not only achieved high accuracy but also delivered transparent decision-making insights through SHAP, aligning clinical utility with algorithmic transparency. Similarly, a comprehensive analysis of AI-driven IoMT (Internet of Medical Things) systems [151] categorized models based on data types, methodologies, and explainability techniques, underscoring the critical role of integrating interpretability tools like LIME and SHAP to address challenges in model transparency and foster trust among healthcare professionals. Building on this, CNN-TumorNet [152] demonstrated exceptional performance (99% accuracy) in classifying brain tumours from MRI scans by embedding the Local Interpretable Model-agnostic Explanations (LIME) framework. This integration enabled clinicians to visualize region-specific tumour features, bridging the gap between complex model outputs and actionable clinical insights. Further advancing cardiovascular disease prediction, the AC2 model [153] synergized CNNs with LightGBM and SHAP, achieving high diagnostic accuracy using the Behavioural Risk Factor Surveillance System (BRFSS) dataset. By quantifying feature contributions, SHAP clarified risk factors driving predictions, ensuring the model's reliability in real-world medical settings. Collectively, these literatures highlight the transformative potential of explainable AI (XAI) in healthcare, where hybrid architectures paired with interpretability techniques enhance diagnostic accuracy, foster clinician confidence, and support ethically grounded decision-making. Future research could further strengthen these frameworks by incorporating advanced statistical methods, such as those proposed by Loh [154] for variable selection in high-dimensional, small-sample (large p , small n) medical datasets. Integrating such techniques with XAI tools like SHAP or LIME may refine model generalizability and interpretability, particularly for sparse or heterogeneous healthcare data, while addressing inherent challenges of dimensionality and overfitting.

6 Drill-Down Analysis of Feature Selection And Hyperparameter-Tuning

This section offers a drill-down analysis of these two methods, focusing on their effects on classification performance, computational complexity, and case studies specific to various diseases. Tables 7, 8, and 9 show the effects of feature selection and hyperparameter-tuning on classification performance, computational complexity and efficiency trade-offs, and disease-specific case studies, respectively [123, 182–186].

Key Observations:

- By eliminating unnecessary or duplicated data, feature selection improves classification performance and helps avoid overfitting.
- To increase accuracy, hyperparameter-tuning optimises model topologies, regularisation strengths, and learning rates.
- Since feature selection guarantees data quality and hyperparameter tweaking provides model performance, combining the two methods frequently produces the best results.
- Feature selection eliminates extraneous data points, which typically lowers computational complexity.
- Hyperparameter optimisation can greatly increase computational load, particularly when used with exhaustive techniques like Grid Search.
- When it comes to hyperparameter tweaking, sophisticated optimisation methods like Bayesian Optimisation and GAs can lower computational costs.

Key Takeaways:

Key Observations:

Table 7 Effect of feature selection and hyperparameter-tuning on classification performance

Aspect	Feature selection	Hyperparameter-tuning
Objective	Identifies the most relevant features for training	Optimizes model parameters for better performance
Effect on accuracy	Can enhance accuracy by removing irrelevant/noisy features	Fine-tunes the model to achieve optimal accuracy
Overfitting risk	Reduces overfitting by eliminating redundant data	Can cause overfitting if tuned aggressively
Generalizability	Improves generalization by focusing on meaningful features	Helps the model adapt better to different datasets
Dimensionality reduction	Decreases the number of input features	Does not impact feature count but affects model complexity

Table 8 Computational complexity and efficiency trade-offs in feature selection and hyperparameter-tuning

Criterion	Feature selection	Hyperparameter-tuning
Computational cost	Low to moderate (depends on feature selection method)	High (especially for deep learning models)
Efficiency	Increases efficiency by reducing feature space	Can slow down training if search space is large
Optimization method	Uses statistical methods (e.g., filter, wrapper, embedded methods)	Requires grid search, random search, or Bayesian optimization
Scalability	Works well with high-dimensional data	Becomes computationally expensive for large datasets

Table 9 Disease-specific case studies on feature selection and hyperparameter-tuning approaches

Disease	Feature selection approach	Hyperparameter-tuning approach	Key findings
Cancer diagnosis (breast, lung, prostate)	RFE with SVM	Bayesian Optimization for SVM parameters	Feature selection improved interpretability, while tuning boosted accuracy (AUC > 0.92)
Diabetes prediction	PCA	Random Search for Neural Network parameters	PCA reduced dimensionality, and tuning enhanced deep learning performance
Cardiovascular disease	Correlation-based Feature Selection (CFS)	Grid Search for RF	Feature reduction improved training speed; hyperparameter-tuning maximized precision
Polycystic ovary syndrome (PCOS)	Mutual Information Selection	Genetic Algorithm for XGBoost tuning	Feature selection eliminated irrelevant data, improving recall for PCOS detection
Alzheimer's disease detection	LASSO Regression for Biomarker Selection	Reinforcement Learning for CNN tuning	Biomarker-based selection improved clinical interpretability, tuning refined CNN efficiency

- Feature selection improves interpretability and lowers noise in illness classification (e.g., biomarker detection in cancer).
- In order to fine-tune deep learning models for high accuracy in complicated datasets, hyperparameter-tuning is essential.
- The best accuracy, effectiveness, and robustness in illness prediction models are obtained by combining the two approaches.

7 Challenges and Future Directions

The integration of hyperparameter-tuning and feature selection in hybrid ML models for biomedical disease classification, while promising, faces significant methodological limitations that hinder clinical adoption. Filter-based feature selection methods, though computationally efficient, often overlook synergistic interactions among biomarkers, leading to suboptimal biomarker identification in complex diseases like cancer or metabolic disorders. Wrapper methods, despite their accuracy, suffer from high computational costs and overfitting risks in small medical datasets, limiting their utility in resource-constrained settings. Embedded techniques such as LASSO regularization struggle with highly correlated clinical variables, producing unstable feature subsets that reduce model reproducibility. Hyperparameter-tuning approaches like GS and RS lack scalability for deep learning architectures, as their exhaustive or random sampling strategies fail to efficiently navigate high-dimensional parameter spaces. Bayesian Optimization, though effective, becomes computationally prohibitive in federated learning environments due to communication overhead, while metaheuristic algorithms (e.g., GA, PSO) introduce non-deterministic outcomes, complicating regulatory compliance. Hybrid frameworks combining these methods often amplify these issues, particularly in real-time clinical systems where latency constraints demand rapid inference. For instance, iterative optimization workflows integrating metaheuristic feature selection with gradient-based hyperparameter-tuning may incur unacceptable delays in critical care scenarios, such as sepsis prediction or cardiac event detection, where timely intervention is paramount.

The optimization landscape for disease classification utilizing hybrid ML algorithms presents a variety of significant challenges that must be addressed for effective implementation [187]. These algorithms integrate hyperparameter-tuning and feature selection to increase their predictive performance [188]. However, the complexity of disease data, the variability of patient populations, and the computational demands of these techniques complicate the classification process. To navigate these challenges, it is essential to delve deeper into the methodologies employed for

hyperparameter-tuning, exploring advanced techniques [8]. The selection of pertinent features is essential for enhancing model accuracy and interpretability. Future study should concentrate on novel methodologies for feature selection that can adjust to the evolving characteristics of medical data. [187]. Moreover, effective validation methods are necessary to confirm that these hybrid models generalize well to unseen data. By focusing on these areas, researchers can create new avenues for enhancing disease classification, hence boosting diagnostic precision and patient outcomes. [189].

Scalability remains a critical barrier, as high-dimensional omics data and multi-modal imaging exacerbate computational complexity. Existing hybrid models frequently prioritize accuracy over operational efficiency, rendering them impractical for deployment on edge devices or embedded systems in point-of-care diagnostics. Furthermore, the lack of standardized benchmarks for evaluating trade-offs between model performance and computational cost complicates objective comparisons across studies. Clinical interpretability is another concern: many hybrid models prioritize algorithmic performance over explainability, producing "black box" predictions that clinician's distrust. For example, ensembles combining recursive feature elimination with deep reinforcement learning for hyperparameter-tuning may achieve high accuracy in oncology but fail to highlight biologically plausible biomarkers, limiting translational impact. As the volume and complexity of this data grow, traditional approaches often struggle to keep pace. This necessitates the development of more efficient algorithms capable of handling large datasets while minimizing processing time and resource consumption. Additionally, the implementation of parallelization techniques becomes essential, allowing computations to be distributed across multiple processing units. This can significantly enhance performance and enable researchers to extract meaningful insights from complex datasets more quickly and effectively [190]. The ability to generalize to unseen datasets is vital for the clinical applicability of any model or methodology. This process requires the development and implementation of robust methodologies that effectively prevent overfitting, which arises when a model implements well on training data but fails to accurately predict outcomes on new, unseen data. To ensure that the model exhibits consistent performance across diverse populations, it is essential to account for various demographic, clinical, and environmental factors during the training process. Achieving this level of generalization will increase the model's reliability and efficacy in real-world clinical settings, ultimately leading to better patient outcomes and a more comprehensive understanding of healthcare trends [191]. Additionally, the intricate nature of hybrid models can significantly impact their explainability and interpretability. This lack of clarity not only makes it challenging for clinicians to understand how these

models arrive at their conclusions but also raises concerns about trust in their recommendations [192]. As a result, widespread adoption in clinical settings may be impeded, particularly in scenarios where transparency is essential for informed decision-making and patient care.

Future research directions should emphasize the integration of emerging methods that have the potential to improve the efficacy of disease classification. One key approach is the utilization of deep learning methods, which can excel at extracting complex features from large datasets, thereby increasing the accuracy of models in identifying various diseases [193]. Additionally, metaheuristic algorithms should be explored for their ability to offer efficient optimization. These algorithms can help fine-tune model parameters and select the most relevant features, ultimately leading to better performance in disease classification tasks. Moreover, the consideration of federated learning is essential, as it allows for collaborative model development across multiple institutions while preserving patient privacy. By enabling different parties to contribute to a common model without sharing sensitive data, federated learning can facilitate the development of robust models that are equipped to handle diverse populations and data sources [194]. Automated hyperparameter-tuning via deep reinforcement learning may reduce reliance on manual expertise but requires rigorous validation to ensure robustness across diverse patient cohorts. For real-time applications, lightweight hybrid architectures leveraging quantization and pruning techniques could balance accuracy with computational efficiency, enabling deployment in wearable devices or ICU monitoring systems. Additionally, explainability-driven optimization—embedding clinical domain knowledge into feature selection criteria and hyperparameter constraints—will enhance model trustworthiness. Finally, community efforts to establish standardized evaluation metrics for scalability, interpretability, and real-time performance are essential to bridge the gap between theoretical advances and clinical implementation [8, 22, 24, 26]. In summary, addressing these obstacles and exploring these advanced methodologies will be critical for pushing the boundaries of ML in the context of disease classification and optimizing the performance of predictive models.

8 Conclusion

The combination of hyperparameter-tuning and feature selection within ML algorithms represents a significant opportunity for enhancing the accuracy and efficiency of disease classification tasks. Hyperparameter-tuning involves optimizing various parameters of the ML model to improve its performance, while feature selection focuses on finding the most appropriate variables that contribute to the model's predictive capability. By effectively integrating these

two processes, researchers can create models that not only achieve higher accuracy in classifying diseases but also reduce computational costs and increase model interpretability. This synergy enables practitioners to develop more robust diagnostic tools that can lead to better healthcare outcomes and more targeted treatment strategies. The key findings underscore the important trade-offs between sequential and simultaneous optimization strategies in the context of data analysis and model development. These findings demonstrate that the selection of an optimization approach must be customized to the distinct attributes of the dataset and the computational resources at hand. For researchers and experts in the field, several practical recommendations emerge from these findings. First, it is crucial to adopt robust cross-validation methods, which help ensure that model performance is reliable and not overly dependent on particular subsets of the data. This approach aids in validating the model's generalizability across different scenarios. Additionally, prioritizing explainable models is essential in facilitating clinical translation. Explainability allows stakeholders, including clinicians and patients, to understand the decision-making process of the algorithm, thus building trust in its predictions and outcomes. Finally, it is vital to carefully evaluate the computational costs linked with complex optimization methods. Understanding these costs will help in making informed decisions about resource allocation, ensuring that the selected optimization approaches are not only effective but also feasible given the constraints of time and computational power. By considering these factors, researchers and practitioners can make more informed choices that enhance the overall quality and applicability of their work. Future research should focus on exploring emerging methods such as deep learning and federated learning. It is important to address challenges related to scalability and generalization while developing hybrid optimization frameworks that balance performance with interpretability. These advancements will lead to more reliable and clinically applicable disease classification models, ultimately improving patient outcomes.

Authors Contributions Sanjay Dhanka was responsible for data curation, resource acquisition, investigation, formal analysis, conceptualization, methodology development, project management, drafting the initial manuscript contents, reviewing and editing the content, and ensuring the validity of the research findings. Abhinav Sharma prepared original draft, collecting raw material, investigation, data curation, formal analysis, reviewing and editing the content of this study. Ankur Kumar contributed to the formal analysis, investigation, writing, reviewing and editing of the content of this study. Surita Maini and Haswant Vundavilli contributed to supervision, validation of references, formal analysis, reviewing and editing of the content of this study during revision.

Funding This study was conducted independently without external financial support.

Data Availability This study conducted a systematic review of intelligent models for diagnosing biomedical disease and has not utilized any datasets.

Declarations

Competing interests The authors declare no competing financial or individual interests related to this study.

Ethical approval Ethical considerations regarding data privacy and usage were not applicable in this study due to the review nature of this study.

References

- Suryadi MK, Herteno R, Saputro SW, Faisal MR, Nugroho RA (2024) Comparative study of various hyperparameter tuning on random forest classification with SMOTE and feature selection using genetic algorithm in software defect prediction. *J Electron Electromed Eng Med Inform* 6(2):137–147. <https://doi.org/10.35882/jeeemi.v6i2.375>
- Yang L, Shami A (2020) On hyperparameter optimization of machine learning algorithms: theory and practice. *Neurocomputing* 415:295–316. <https://doi.org/10.1016/j.neucom.2020.07.061>
- Amou MA, Xia K, Kamhi S, Mouhafid M (2022) A novel MRI diagnosis method for brain tumor classification based on CNN and Bayesian optimization. *Healthcare* 10(3):1–21. <https://doi.org/10.3390/healthcare10030494>
- Abawajy J, Darem A, Alhashmi AA (2021) Feature subset selection for malware detection in smart iot platforms. *Sensors (Switzerland)* 21(4):1–19. <https://doi.org/10.3390/s21041374>
- Dasgupta S, Huang Y (2020) Selecting biomarkers for building optimal treatment selection rules by using kernel machines. *J R Stat Soc Ser C* 69(1):69–88. <https://doi.org/10.1111/rssc.12379>
- LaCasse PM, Otieno W, Maturana FP (2018) A hierarchical, fuzzy inference approach to data filtration and feature prioritization in the connected manufacturing enterprise. *J Big Data*. <https://doi.org/10.1186/s40537-018-0155-2>
- U. M. O. Techniques, “fakultät statistik Multi-objective Analysis of Machine Learning Algorithms Using Model-based Optimization Techniques,” no. January 2019.
- Ahmed Z, Mohamed K, Zeeshan S, Dong XQ (2020) Artificial intelligence with multi-functional machine learning platform development for better healthcare and precision medicine. *Database* 2020:1–35. <https://doi.org/10.1093/database/baaa010>
- Kline A et al (2022) Multimodal machine learning in precision health: a scoping review. *Npj Digit Med*. <https://doi.org/10.1038/s41746-022-00712-8>
- Mashrafi AM (2022) Current trends on the application of artificial intelligence in medical sciences. *Bioinformation* 18(11):1050–1061. <https://doi.org/10.6026/973206300181050>
- Schaefer J, Lehne M, Schepers J, Prasser F, Thun S (2020) The use of machine learning in rare diseases: a scoping review. *Orphanet J Rare Dis* 15(1):1–10. <https://doi.org/10.1186/s13023-020-01424-6>
- Hassan BM, Elagamy SM (2025) Personalized medical recommendation system with machine learning. *Neural Comput Appl* 37(9):6431–6447. <https://doi.org/10.1007/s00521-024-10916-6>
- Alum EU, Ugwu OPC (2025) Artificial intelligence in personalized medicine: transforming diagnosis and treatment. *Discov Appl Sci*. <https://doi.org/10.1007/s42452-025-06625-x>
- Al-Zaiti SS et al (2022) A clinician’s guide to understanding and critically appraising machine learning studies: a checklist for Ruling Out Bias Using Standard Tools in Machine Learning (ROBUST-ML). *Eur Hear J Digit Health* 3(2):125–140. <https://doi.org/10.1093/ehjdh/ztac016>
- Oikonomou EK, Khera R (2023) Machine learning in precision diabetes care and cardiovascular risk prediction. *Cardiovasc Diabetol* 22(1):1–16. <https://doi.org/10.1186/s12933-023-01985-3>
- Banjar H, Adelson D, Brown F, Chaudhri N (2017) Intelligent techniques using molecular data analysis in leukaemia: an opportunity for personalized medicine support system. *Biomed Res Int*. <https://doi.org/10.1155/2017/3587309>
- Marques L et al (2024) Advancing precision medicine: a review of innovative in silico approaches for drug development, clinical pharmacology and personalized healthcare. *Pharmaceutics*. <https://doi.org/10.3390/pharmaceutics16030332>
- Potharlanka JL, Nirupama Bhat M (2024) Feature importance feedback with Deep Q process in ensemble-based metaheuristic feature selection algorithms. *Sci Rep* 14(1):1–30. <https://doi.org/10.1038/s41598-024-53141-w>
- Trindade AR, Campelo F (2019) Tuning metaheuristics by sequential optimisation of regression models. *Appl Soft Comput J*. <https://doi.org/10.1016/j.asoc.2019.105829>
- Liu C, Zhang Y (2021) Hyper-parameter optimization based on soft actor critic and hierarchical mixture regularization, pp 1–17
- Singh J, Pandey B, Karna S, Arora AS, Kumar A (2024) Enhancing the thermographic diagnosis of maxillary sinusitis using deep learning approach. *Quant Infrared Thermogr J* 00(00):1–15. <https://doi.org/10.1080/17686733.2024.2349976>
- Wojciuk M, Swiderska-Chadaj Z, Siwek K, Gertych A (2024) Improving classification accuracy of fine-tuned CNN models: Impact of hyperparameter optimization. *Heliyon* 10(5):e26586. <https://doi.org/10.1016/j.heliyon.2024.e26586>
- Xin J, Khishe M, Zeebaree DQ, Abualigah L, Ghazal TM (2024) Adaptive habitat biogeography-based optimizer for optimizing deep CNN hyperparameters in image classification. *Heliyon* 10(7):e28147. <https://doi.org/10.1016/j.heliyon.2024.e28147>
- Senagi K, Jouandeau N (2018) A non-deterministic strategy for searching optimal number of trees hyperparameter in Random Forest. In: *Proceedings of 2018 Federated Conference on Computer Science Information System FedCSIS 2018*, pp 73–80. <https://doi.org/10.15439/2018F202>
- Khodak M et al. (2021) Federated hyperparameter tuning: challenges, baselines, and connections to weight-sharing. In: *Advances in Neural Information Processing System*, vol 23, no. NeurIPS, pp 19184–19197
- Uddin MF, Lee J, Rizvi S, Hamada S (2018) Proposing enhanced feature engineering and a selection model for machine learning processes. *Appl Sci*. <https://doi.org/10.3390/app8040646>
- Raja K, Patrick M, Gao Y, Madu D, Yang Y, Tsoi LC (2017) A review of recent advancement in integrating omics data with literature mining towards biomedical discoveries. *Int J Genom*. <https://doi.org/10.1155/2017/6213474>
- Kamalov F, Sulieman H, Moussa S, Reyes JA, Safaraliev M (2023) Nested ensemble selection: an effective hybrid feature selection method. *Heliyon* 9(9):e19686. <https://doi.org/10.1016/j.heliyon.2023.e19686>
- Li J, Liu H (2017) Challenges of feature selection for big data analytics. *IEEE Intell Syst* 32(2):9–15. <https://doi.org/10.1109/MIS.2017.38>
- S. A. Almutairi, “An Optimized Feature Selection and Hyperparameter Tuning Framework for Automated Heart Disease Diagnosis,” 2023, <https://doi.org/10.32604/csse.2023.041609>.
- Kaur BP, Singh H, Hans R, Sharma SK, Sharma C, Hassan MM (2024) A Genetic algorithm aided hyper parameter optimization based ensemble model for respiratory disease prediction with Explainable AI. *PLoS ONE* 19(12):e0308015

32. Kumar A, Singh J, Khan AA (2024) A comprehensive machine learning framework with particle swarm optimization for improved polycystic ovary syndrome (PCOS) diagnosis. *Eng Res Express*. <https://doi.org/10.1088/2631-8695/ad76f9>
33. Sharma A, Dhanka S, Kumar A, Nain M, Dhanka B (2025) A systematic review on machine learning intelligent systems for heart disease diagnosis. *Arch Comput Methods Eng*. <https://doi.org/10.1007/s11831-025-10271-2>
34. Zhu X, Yang Y (2015) Variable selection after screening : with or without data splitting ?, pp 191–203. <https://doi.org/10.1007/s00180-014-0528-8>.
35. Ebrahimi Z, Loni M, Daneshlab M, Gharehbaghi A (2020) A review on deep learning methods for ECG arrhythmia classification. *Expert Syst Appl* 17:100033. <https://doi.org/10.1016/j.eswax.2020.100033>
36. Zhou Y et al (2023) A privacy-preserving logistic regression-based diagnosis scheme for digital healthcare. *Futur Gener Comput Syst* 144:63–73. <https://doi.org/10.1016/j.future.2023.02.022>
37. Sharma A, Dhanka S, Kumar A, Maini S (2024) A comparative study of heterogeneous machine learning algorithms for arrhythmia classification using feature selection technique and multi-dimensional datasets. *Eng Res Express* 6(3):035209. <https://doi.org/10.1088/2631-8695/ad5d51>
38. Dhanka S, Bhardwaj VK, Maini S (2023) Comprehensive analysis of supervised algorithms for coronary artery heart disease detection. *Expert Syst* 40(7):1–21. <https://doi.org/10.1111/essy.13300>
39. Bentéjac C, Csörgő A, Martínez-Muñoz G (2021) A comparative analysis of gradient boosting algorithms. *Artif Intell Rev* 54(3):1937–1967. <https://doi.org/10.1007/s10462-020-09896-5>
40. Gredell DA et al (2019) Comparison of machine learning algorithms for predictive modeling of beef attributes using rapid evaporative ionization mass spectrometry (REIMS) data. *Sci Rep* 9(1):5721. <https://doi.org/10.1038/s41598-019-40927-6>
41. Torlay L, Perrone-Bertolotti M, Thomas E, Baciú M (2017) Machine learning–XGBoost analysis of language networks to classify patients with epilepsy. *Brain Inform* 4(3):159–169. <https://doi.org/10.1007/s40708-017-0065-7>
42. Mo H, Sun H, Liu J, Wei S (2019) Developing window behavior models for residential buildings using XGBoost algorithm. *Energy Build* 205:109564. <https://doi.org/10.1016/j.enbuild.2019.109564>
43. Yu D, Xiang B (2023) Discovering topics and trends in the field of artificial intelligence: using LDA topic modelling. *Expert Syst Appl* 225:120114
44. Wu L, Shen C, van den Hengel A (2017) Deep linear discriminant analysis on fisher networks: a hybrid architecture for person re-identification. *Pattern Recogn* 65:238–250. <https://doi.org/10.1016/j.patcog.2016.12.022>
45. Ampomah EK, Nyame G, Qin Z, Addo PC, Gyamfi EO, Gyan M (2021) Stock market prediction with Gaussian Naïve Bayes machine learning algorithm. *Informatica* 45(2):243–256. <https://doi.org/10.31449/inf.v45i2.3407>
46. Shen M, Tang X, Zhu L, Du X, Guizani M (2019) Privacy-preserving support vector machine training over blockchain-based encrypted IoT data in smart cities. *IEEE Internet Things J* 6(5):7702–7712. <https://doi.org/10.1109/JIOT.2019.2901840>
47. Xu G, Liu M, Jiang Z, Söfker D, Shen W (2019) Bearing fault diagnosis method based on deep convolutional neural network and random forest ensemble learning. *Sensors* 19(5):1088. <https://doi.org/10.3390/s19051088>
48. Prasojo RA et al (2023) Precise transformer fault diagnosis via random forest model enhanced by synthetic minority over-sampling technique. *Electr Power Syst Res* 220:109361. <https://doi.org/10.1016/j.epsr.2023.109361>
49. Lan T, Hu H, Jiang C, Yang G, Zhao Z (2020) A comparative study of decision tree, random forest, and convolutional neural network for spread-F identification. *Adv Sp Res* 65(8):2052–2061. <https://doi.org/10.1016/j.asr.2020.01.036>
50. McCulloch WS, Pitts W (1943) A logical calculus of the ideas immanent in nervous activity. *Bull Math Biophys* 52:115–133
51. Mohamad ET, Faradonbeh RS, Armaghani DJ, Monjezi M, Majid MZA (2017) An optimized ANN model based on genetic algorithm for predicting ripping production. *Neural Comput Appl* 28(s1):393–406. <https://doi.org/10.1007/s00521-016-2359-8>
52. Singh TN, Kanchan R, Verma AK, Saigal K (2005) A comparative study of ANN and Neuro-fuzzy for the prediction of dynamic constant of rockmass. *J Earth Syst Sci* 114(1):75–86. <https://doi.org/10.1007/BF02702010>
53. Hajihassani M, Armaghani DJ, Sohaei H, Mohamad ET, Marto A (2014) Prediction of airblast-overpressure induced by blasting using a hybrid artificial neural network and particle swarm optimization. *Appl Acoust* 80:57–67
54. Aslan SN, Özalp R, Uçar A, Güzelci C (2022) New CNN and hybrid CNN-LSTM models for learning object manipulation of humanoid robots from demonstration. *Clust Comput* 25(3):1575–1590. <https://doi.org/10.1007/s10586-021-03348-7>
55. Lanjewar MG, Parab JS, Shaikh AY, Sequeira M (2023) CNN with machine learning approaches using ExtraTreesClassifier and MRMR feature selection techniques to detect liver diseases on cloud. *Clust Comput* 26(6):3657–3672. <https://doi.org/10.1007/s10586-022-03752-7>
56. Lanjewar MG, Gurav OL (2022) Convolutional neural networks based classifications of soil images. *Multimed Tools Appl* 81(7):10313–10336. <https://doi.org/10.1007/s11042-022-12200-y>
57. Lanjewar MG, Morajkar PP, Parab J (2022) Detection of tartrazine colored rice flour adulteration in turmeric from multi-spectral images on smartphone using convolutional neural network deployed on PaaS cloud. *Multimed Tools Appl* 81(12):16537–16562. <https://doi.org/10.1007/s11042-022-12392-3>
58. Bilalli B, Abell A (2017) Intelligent assistance for data pre-processing
59. Chicco D, Oneto L, Tavazzi E (2022) Eleven quick tips for data cleaning and feature engineering. *PLoS Comput Biol* 18(12):e1010718
60. Ahangaran M, Zhu H, Li R, Yin L, Jang J, Chaudhry AP (2024) DREAMER : a computational framework to evaluate readiness of datasets for machine learning. *BMC Med Inform Decis Mak*. <https://doi.org/10.1186/s12911-024-02544-w>
61. Ayar M, Isazadeh A, Gharehchopogh FS, Seyedi M (2023) NSICA: Multi-objective imperialist competitive algorithm for feature selection in arrhythmia diagnosis. *Comput Biol Med* 161:7025. <https://doi.org/10.1016/j.combiomed.2023.107025>
62. Heydarian M, Doyle TE, Samavi R (2022) MLCM: multi-label confusion matrix. *IEEE Access* 10:19083–19095. <https://doi.org/10.1109/ACCESS.2022.3151048>
63. Kuzudisli C, Bakir-Gungor B, Bulut N, Qaqish B, Yousef M (2023) Review of feature selection approaches based on grouping of features. *PeerJ*. <https://doi.org/10.7717/peerj.15666>
64. Al-Dhaheri SA (2021) CUNY academic works a new feature selection method based on class association rule how does access to this work benefit you ? Let us know ! by
65. Kumar GS et al (2025) SRADHO: statistical reduction approach with deep hyper optimization for disease classification using artificial intelligence. *Sci Rep* 15(1):1–27. <https://doi.org/10.1038/s41598-024-82838-1>
66. Li Y, Mansmann U, Du S, Hornung R (2022) Benchmark study of feature selection strategies for multi-omics data. *BMC Bioinform* 23(1):1–18. <https://doi.org/10.1186/s12859-022-04962-x>

67. Zhang Y, Lin Z, Kwoh CK (2020) Information theory-based feature selection: Minimum distribution similarity with removed redundancy, vol 12141 LNCS. Springer. https://doi.org/10.1007/978-3-030-50426-7_1.
68. Yang J, Qu Z, Liu Z (2014) Improved feature-selection method considering the imbalance problem in text categorization. *Sci World J*. <https://doi.org/10.1155/2014/625342>
69. Inquires QM Visualization of the significant explicative non-symmetrical correspondence analysis, pp 64–72
70. Amin HA (2015) Determination of single factor fixed effect experiments in CRD with multiple linear regression analysis 5(13):52–66
71. Baraldi P, Zio E, Roverso D (2007) Feature selection for nuclear transient diagnostics. In: Halden Work Report, HWR-828, OECD Halden Reactor Project, 2007, pp 1–46
72. Dash M, Liu H (1997) Feature selection for classification. *Intell Data Anal* 1(3):131–156. <https://doi.org/10.3233/IDA-1997-1302>
73. Kursu MB (2014) Robustness of random forest-based gene selection methods. *BMC Bioinform*. <https://doi.org/10.1186/1471-2105-15-8>
74. Ding W, Lin CT, Pedrycz W (2020) Multiple relevant feature ensemble selection based on multilayer co-evolutionary consensus mapreduce. *IEEE Trans Cybern* 50(2):425–439. <https://doi.org/10.1109/TCYB.2018.2859342>
75. Correia A, Lecue F, Correia A, Lecue F, H. F. Selection, and T. A. Conference (2019) Human-in-the-loop feature selection to cite this version : HAL Id : hal-01934916 human-in-the-loop feature selection
76. Li J et al (2017) Feature selection: a data perspective. *ACM Comput Surv*. <https://doi.org/10.1145/3136625>
77. Finch WH, Finch MEH (2016) Regularization methods for fitting linear models with small sample sizes: Fitting the lasso estimator using R. *Pract Assess Res Eval*. <https://doi.org/10.7275/jr3d-cq04>
78. Chaudhry MU, Yasir M, Asghar MN, Lee JH (2020) Monte carlo tree search-based recursive algorithm for feature selection in high-dimensional datasets. *Entropy* 22(10):1–15. <https://doi.org/10.3390/e22101093>
79. Pudjihartono N, Fadason T, Kempa-Liehr AW, O'Sullivan JM (2022) A review of feature selection methods for machine learning-based disease risk prediction. *Front Bioinform* 2:1–17. <https://doi.org/10.3389/fbinf.2022.927312>
80. Bencsik B, Reményi I, Szemenyei M, Botzheim J (2023) Designing an embedded feature selection algorithm for a drowsiness detector model based on electroencephalogram data. *Sensors*. 23(4):1874
81. Basir MA, Yusof Y, Hussin MS (2019) Optimization of attribute selection model using bio-inspired algorithms. *J Inf Commun Technol* 18(1):35–55
82. Alvarez-Alvarado MS, Alban-Chacón FE, Lamilla-Rubio EA, Rodríguez-Gallegos CD, Velásquez W (2021) Three novel quantum-inspired swarm optimization algorithms using different bounded potential fields. *Sci Rep* 11(1):1–22. <https://doi.org/10.1038/s41598-021-90847-7>
83. Aljamel A, Osman T, Acampora G, Vitiello A, Zhang Z (2019) Smart information retrieval: domain knowledge centric optimization approach. *IEEE Access* 7:4167–4183. <https://doi.org/10.1109/ACCESS.2018.2885640>
84. Freitas D, Lopes LG, Morgado-Dias F (2020) Particle swarm optimisation: a historical review up to the current developments. *Entropy* 22(3):1–36. <https://doi.org/10.3390/E22030362>
85. Saraç E, Özel SA (2014) An ant colony optimization based feature selection for web page classification. *Sci World J*. <https://doi.org/10.1155/2014/649260>
86. Veric CS (2006) On the potential of failure. *Am Q* 58(1):255–264. <https://doi.org/10.1353/aq.2006.0035>
87. Kahira AN, Gomez LB, Conejero J, Badia RM (2019) Accelerating hyperparameter optimisation with PyCOMPSs. *ACM Int Conf Proc Ser*. <https://doi.org/10.1145/3339186.3339200>
88. Demšar J, Zupan B (2021) Hands-on training about overfitting. *PLoS Comput Biol* 17(3):1–19. <https://doi.org/10.1371/journal.pcbi.1008671>
89. Mohammed RO, Cawley GC (2017) Over-fitting in model selection with Gaussian process regression. *Lecture Notes in Computer Science (Including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, vol 10358 LNAI, no. 2013, pp 192–205. https://doi.org/10.1007/978-3-319-62416-7_14.
90. Steinert S et al (2024) A refined approach for evaluating small datasets via binary classification using machine learning. *PLoS ONE* 19:1–21. <https://doi.org/10.1371/journal.pone.0301276>
91. Yang L, Shami A (2020) Neurocomputing on hyperparameter optimization of machine learning algorithms: theory and practice. *Neurocomputing* 415:295–316. <https://doi.org/10.1016/j.neucom.2020.07.061>
92. Stubbs R, Wilson K, Rostami S (2020) Hyper-parameter optimisation by restrained stochastic hill climbing. *Adv Intell Syst Comput* 1043:189–200. https://doi.org/10.1007/978-3-030-29933-0_16
93. Shang X et al (2019) A simple dynamic bandit algorithm for hyper-parameter tuning to cite this version.: In: Workshop on Automated Machine Learning at International Conference on Machine Learning, HAL Id: hal-02145200
94. Hourdin F et al (2017) The art and science of climate model tuning. *Bull Am Meteorol Soc* 98(3):589–602. <https://doi.org/10.1175/BAMS-D-15-00135.1>
95. Joy G, Huyck C, Yang XS (2023) Review of parameter tuning methods for nature-inspired algorithms. *Benchmarks Hybrid Algorithms Optim Appl* 22:33–47
96. Felzmann H, Fosch-Villaronga E, Lutz C, Tamò-Larrieux A (2020) Towards transparency by design for artificial intelligence. *Sci Eng Ethics* 26(6):3333–3361. <https://doi.org/10.1007/s11948-020-00276-4>
97. Karcher T, Tichy WF (2014) Autotuning and self-adaptability in concurrency libraries. *No. Racing*
98. Kangasräsio A, Jokinen JPP, Oulasvirta A, Howes A, Kaski S (2019) Parameter inference for computational cognitive models with approximate Bayesian computation. *Cogn Sci*. <https://doi.org/10.1111/cogs.12738>
99. Mehdiy A, Chehri A, Jakimi A, Saadane R (2024) Hyperparameter optimization with genetic algorithms and XGBoost: a step forward in smart grid fraud detection. *Sensors*. <https://doi.org/10.3390/s24041230>
100. Erden C, Demir HI, Kökçam AH (2023) Enhancing machine learning model performance with hyper parameter optimization: a comparative study. <https://doi.org/10.48550/arXiv.2302.11406>.
101. Fu W, Nair V, Menzies T (2017) Why is differential evolution better than grid search for tuning defect predictors?. *arXiv preprint arXiv:1609.02613*
102. Han C, Feng Y, Wang T (2023) From random search to bandit learning in metric measure spaces. *arXiv preprint arXiv:2305.11509*
103. Bartz E, Zaefferer M, Mersmann O, Bartz-Beielstein T (2021) Experimental investigation and evaluation of model-based hyperparameter optimization. [Online]. <http://arxiv.org/abs/2107.08761>
104. Florea AC, Andonie R (2019) Weighted random search for hyperparameter optimization. <https://doi.org/10.48550/arXiv.2004.01628>.
105. Zabinsky ZB, Linz DD (2023) Hesitant adaptive search with estimation and quantile adaptive search for global optimization with noise. *J Glob Optim* 87(1):31–55. <https://doi.org/10.1007/s10898-023-01307-7>

106. Chen J, Tseng YJ (2022) A general optimization protocol for molecular property prediction using a deep learning network. *Brief Bioinform* 23(1):1–13. <https://doi.org/10.1093/bib/bbab367>
107. Hossain MR, Timmer D (2021) Machine learning model optimization with hyper parameter tuning approach. *Glob J Comput Sci Technol* D 21(2):31
108. Kotthoff L, Johnson PA (2021) Bayesian optimization in materials science : a survey, pp 1–15
109. Li YL, Rudner TGJ, Wilson AG (2023) A study of Bayesian neural network surrogates for bayesian optimization. <https://doi.org/10.48550/arXiv.2305.20028>
110. Olivares R et al (2022) Entropy-based diversification approach for bio-computing methods. *Entropy* 24(9):1293. <https://doi.org/10.3390/e24091293>
111. Parsa M, Schuman CD, Date P, Rose DC, Kay B, Mitchell JP, Young SR, Dellana R, Severa W, Potok TE, Roy K (2020) Hyperparameter optimization in binary communication networks for neuromorphic deployment. In: 2020 International Joint Conference on Neural Networks (IJCNN), pp 1–9. IEEE
112. Dereich S, Jentzen A, Riekert A (2024) Learning rate adaptive stochastic gradient descent optimization methods: numerical simulations for deep learning methods for partial differential equations and convergence analyses. *arXiv:2406.14340*
113. Bottou L (2010) Large-scale machine learning with stochastic gradient descent. In: *Proceedings of COMPSTAT'2010: 19th International Conference on Computational Statistics* Paris France, August 22–27, 2010 Keynote, Invited and Contributed Papers 2010, pp 177–186. Physica-Verlag HD
114. Dauphin YN, De Vries H, Bengio Y Equilibrated adaptive learning rates for non-convex optimization, pp 1–10
115. Choromanska A, Henaff M, Mathieu M (2015) The loss surfaces of multilayer networks, vol 38
116. Sh M, Mohammad M, Hani S, Al Mimi M, Abualigah L, Zitar RA (2023) Gradient-based optimizer (GBO): a review, theory, variants, and applications. *Arch Comput Methods Eng* 30(4):2431–2449. <https://doi.org/10.1007/s11831-022-09872-y>
117. Sohrabi M, Fathollahi-Fard AM, Gromov VA (2024) Genetic engineering algorithm (GEA): an efficient metaheuristic algorithm for solving combinatorial optimization problems. *Autom Remote Control* 85(3):252–62
118. Shopova EG, Vakliava-Bancheva NG (2006) BASIC—a genetic algorithm for engineering problems solution. *Comput Chem Eng* 30(8):1293–309
119. Li K, Xiang Z, Tan KC (2019) Which surrogate works for empirical performance modelling? A case study with differential evolution. In: 2019 IEEE Congress on Evolutionary Computation (CEC), pp 1988–1995. IEEE
120. Wang Q, Spronck P, Tracht R (2003) An overview of genetic algorithms applied to control engineering problems. In: *Proceedings of the 2003 International Conference on Machine Learning and Cybernetics (IEEE Cat. No. 03EX693)*, vol 3, pp 1651–1656. IEEE
121. Li S, Chen W, Jain S, Jung D, Lee J (2024) Optimization of flow behavior models by genetic algorithm: a case study of aluminum alloy. *J Mater Res Technol* 31:3349–3363. <https://doi.org/10.1016/j.jmrt.2024.07.048>
122. Huang Y, Liu C, Li W, Liu X, Wu JH, Ma F (2024) A multi-functional metastructure with energy dissipation and low-frequency sound-absorption optimized for decoupling by genetic algorithm. *Thin-Walled Struct* 199:111815. <https://doi.org/10.1016/j.tws.2024.111815>
123. Bakır H, Ceviz Ö (2025) Empirical enhancement of intrusion detection systems : a comprehensive approach with genetic algorithm-based hyperparameter tuning and hybrid feature selection. *Arab J Sci Eng* 49(9):13025–13043. <https://doi.org/10.1007/s13369-024-08949-z>
124. Mani NSK, Tirumanadham K, Thaiyalnayaki S, Sriram M (2024) Improving predictive performance in e-learning through hybrid 2-tier feature selection and hyper parameter—optimized 3-tier ensemble modeling. *Int J Inf Technol* 16(8):5429–5456. <https://doi.org/10.1007/s41870-024-02038-y>
125. Abdel-salam M (2024) A proposed framework for crop yield prediction using hybrid feature selection approach and optimized machine learning. *Neural Comput Appl* 36(33):20723–20750. <https://doi.org/10.1007/s00521-024-10226-x>
126. Konak F, Akif M (2024) Feature selection and hyperparameters optimization employing a hybrid model based on genetic algorithm and artificial neural network: forecasting dividend payout, pp 1673–1693. <https://doi.org/10.1007/s10614-023-10530-z>
127. Qaraad M, Amjad S, Manhrawy II, Fathi H, Hassan BA, El Kafrawy P (2021) A hybrid feature selection optimization model for high dimension data classification. *IEEE Access* 9:42884–42895
128. Zeng X, Luo G (2017) Progressive sampling-based bayesian optimization for efficient and automatic machine learning model selection. *Health Inf Sci Syst* 5:1–21
129. Syed AR (2011) A review of cross validation and adaptive model selection
130. Azevedo BF, Maria A, Ana ACR (2024) Hybrid approaches to optimization and machine learning methods : a systematic literature review, vol 113, No. 7. Springer. <https://doi.org/10.1007/s10994-023-06467-x>.
131. Chen J, Qi X, Chen L, Chen F, Cheng G (2020) Knowledge-based systems quantum-inspired ant lion optimized hybrid k-means for cluster analysis and intrusion detection. *Knowl-Based Syst* 203:106167. <https://doi.org/10.1016/j.knosys.2020.106167>
132. El Shorbagy MA, Mousa AYAAA, El Desoky IM (2019) An enhanced genetic algorithm with new mutation for cluster analysis, vol 34 No. 3. Springer, Berlin
133. Pacifico LDS (2021) An evaluation of k-means as a local search operator in hybrid memetic group search optimization for data clustering. *Nat Comput* 20(3):611–636. <https://doi.org/10.1007/s11047-020-09809-z>
134. H. A. Atabay, “A Clustering Algorithm Based on Integration of,” pp. 59–63, 2016.
135. Ratanavilisagul C (2020) Modification fitness function of particle swarm optimization to improve the cluster centroid. In: *IOP Conference Series: Materials Science and Engineering*; vol 965, No. 1, p 012038. IOP Publishing
136. Nguyen TPQ, Kuo RJ (2019) Automatic fuzzy clustering using non-dominated sorting particle swarm optimization algorithm for categorical data. *IEEE Access* 7:99721–99734. <https://doi.org/10.1109/ACCESS.2019.2927593>
137. Thalamala RC, Venkata Swamy Reddy A, Janet B (2019) A novel bio-inspired algorithm based on social spiders for improving performance and efficiency of data clustering. *J Intell Syst* 29(1):311–326
138. M. Niknam and P. Thulasiraman, “LPR : A bio-inspired intelligent learning path recommendation system based on meaningful learning theory,” pp. 3797–3819, 2020.
139. P. Edwin Dhas and B. Sankara Gomathi, “A novel clustering algorithm by clubbing GHFCM and GWO for microarray gene data,” *J. Supercomput.*, vol. 76, no. 8, pp. 5679–5693, 2020, <https://doi.org/10.1007/s11227-019-02953-z>.
140. Singh T (2021) A novel data clustering approach based on whale optimization algorithm. *Expert Syst* 38(3):1–25. <https://doi.org/10.1111/exsy.12657>
141. Bida I, Aouat S (2021) Swarm intelligence-based decision trees induction for classification—a brief analysis. In: 2020 2nd International Workshop on Human-Centric Smart Environments for Health and Well-being (IHSH), pp 165–170. IEEE

142. Jariyavajee C, Polvichai J, Sirinaovakul B (2019) Searching for splitting criteria in multivariate decision tree using adapted jade optimization algorithm. In: 2019 IEEE Symposium Series on Computational Intelligence (SSCI), pp 2534–2540. IEEE
143. Jain R, Batra J, Kar AK, Agrawal H, Tikkiwal VA (2022) A hybrid bio-inspired computing approach for buzz detection in social media. *Evol Intell* 15(1):349–367. <https://doi.org/10.1007/s12065-020-00512-7>
144. Lee CY, Huang KY, Shen YX, Lee YC (2020) Improved weighted k-nearest neighbor based on PSO for wind power system state recognition. *Energies*. <https://doi.org/10.3390/en13205520>
145. Farrell M, Ramadhani KN, Suyanto S (2020) Combined firefly algorithm-random forest to classify autistic spectrum disorders. In: 2020 3rd International Seminar on Research of Information Technology and Intelligent Systems (ISRITI), pp 505–508. IEEE
146. Aslam N et al (2021) Predicting diabetic patient hospital readmission using optimized random forest and firefly evolutionary algorithm. *Int J Adv Sci Eng Inf Technol* 11(5):1876–1883. <https://doi.org/10.18517/ijaseit.11.5.14221>
147. Moldovan D, Anghel I, Cioara T, Salomie I, Chifu V, Pop C (2019) Kangaroo Mob Heuristic for Optimizing Features Selection in Learning the Daily Living Activities of People with Alzheimer's. In: 2019 22nd International Conference on Control Systems and Computer Science (CSCS), pp 236–243. IEEE
148. Oliva D Artificial intelligence for COVID-19
149. Tharwat A, Gabel T, Gabel T (2020) Parameters optimization of support vector machines for imbalanced data using social ski driver algorithm. *Neural Comput Appl* 32(11):6925–6938. <https://doi.org/10.1007/s00521-019-04159-z>
150. Wani NA, Kumar R, Bedi J (2024) DeepXplainer: an interpretable deep learning based approach for lung cancer detection using explainable artificial intelligence. *Comput Methods Programs Biomed* 243:107879. <https://doi.org/10.1016/j.cmpb.2023.107879>
151. Wani NA, Kumar R, Bedi J, Rida I (2024) Explainable AI-driven IoMT fusion: unravelling techniques, opportunities, and challenges with explainable AI in healthcare. *Inf Fusion* 110:102472
152. Rasool N et al (2025) CNN-TumorNet: leveraging explainability in deep learning for precise brain tumor diagnosis on MRI images. *Front Oncol* 15:1–11. <https://doi.org/10.3389/fonc.2025.1554559>
153. Wani NA, Bedi J, Kumar R, Khan MA, Rida I (2024) Synergizing fusion modelling for accurate cardiac prediction through explainable artificial intelligence. *IEEE Trans Consum Electron*. <https://doi.org/10.1109/TCE.2024.3419814>
154. Loh WY (2012) Variable selection for classification and regression in large p, small n problems. *Probability approximations and beyond*. Springer, New York, pp 135–159
155. Zhou Y, Wu H, Luo Q, Abdel-baset M (2019) Knowledge-based systems automatic data clustering using nature-inspired symbiotic organism search algorithm. *Knowl-Based Syst* 163:546–557. <https://doi.org/10.1016/j.knosys.2018.09.013>
156. Alrosan A et al (2021) Automatic data clustering based mean best artificial bee colony algorithm. *Comput Mater Contin* 68(2):1575–1593. <https://doi.org/10.32604/cmc.2021.015925>
157. Kuo RJ, Amornnikun P, Nguyen TPQ (2020) Metaheuristic-based possibilistic multivariate fuzzy weighted c-means algorithms for market segmentation. *Appl Soft Comput J* 96:106639. <https://doi.org/10.1016/j.asoc.2020.106639>
158. Nemmich MA, Debbat F, Slimane M (2019) A data clustering approach using bees algorithm with a memory scheme, vol 50. Springer. https://doi.org/10.1007/978-3-319-98352-3_28
159. Singh H, Kumar Y (2020) A neighborhood search based cat swarm optimization algorithm for clustering problems. *Evol Intell* 13(4):593–609. <https://doi.org/10.1007/s12065-020-00373-0>
160. Eesa AS, Orman Z (2020) A new clustering method based on the bio-inspired cuttlefish optimization algorithm. *Expert Syst* 37(2):e12478
161. Ezugwu AES, Agbaje MB, Aljojo N, Els R, Chiroma H, Elaziz MA (2020) A comparative performance study of hybrid firefly algorithms for automatic data clustering. *IEEE Access* 8:121089–121118. <https://doi.org/10.1109/ACCESS.2020.3006173>
162. Dhal KG, Gálvez J, Das S (2020) Toward the modification of flower pollination algorithm in clustering-based image segmentation. *Neural Comput Appl* 32(8):3059–3077. <https://doi.org/10.1007/s00521-019-04585-z>
163. Dutta D, Sil J, Dutta P (2019) Automatic clustering by multi-objective genetic algorithm with numeric and categorical features. *Expert Syst Appl* 137:357–379. <https://doi.org/10.1016/j.eswa.2019.06.056>
164. Zhang X, Lin Q, Mao W, Liu S, Dou Z, Liu G (2021) Hybrid particle swarm and grey wolf optimizer and its application to clustering optimization. *Appl Soft Comput* 101:107061. <https://doi.org/10.1016/j.asoc.2020.107061>
165. Adibi MA (2019) Single and multiple outputs decision tree classification using bi-level discrete-continues genetic algorithm. *Pattern Recogn Lett* 128:190–196. <https://doi.org/10.1016/j.patrec.2019.09.001>
166. Damanik IS, Windarto AP, Wanto A, Poningsih, Andani SR, Saputra W (2019) Decision tree optimization in C4.5 algorithm using genetic algorithm. *J Phys* 1255(1):12012
167. Nagra AA et al (2020) Hybrid self-inertia weight adaptive particle swarm optimisation with local search using C4.5 decision tree classifier for feature selection problems. *Conn Sci* 32(1):16–36. <https://doi.org/10.1080/09540091.2019.1609419>
168. Sarala SPK, Chitraivel M, Raj RSP (2021) Decision tree based salp swarm optimization for multi medical data classification with feature reduction technique. *Braz Arch Biol Technol*. <https://doi.org/10.1590/1678-4324-2021210240>
169. Shih YH, Ting CK (2019) Evolutionary optimization on k-nearest neighbors classifier for imbalanced datasets. In: 2019 IEEE Congress on Evolution Computer CEC 2019—Proceedings, pp 3348–3355. <https://doi.org/10.1109/CEC.2019.8789921>
170. K. Huang, “Enhancing k -Nearest Neighbors through Learning Transformation Functions by Genetic Programming,” no. 1, pp. 1891–1897, 1891.
171. R. Mittal, V. Malik, V. Singh, J. Singh, and A. Kaur, “Integrating genetic algorithm with random forest for improving the classification performance of web log data,” *PDGC 2020 - 2020 6th Int. Conf. Parallel, Distrib. Grid Comput.*, pp. 177–181, 2020, <https://doi.org/10.1109/PDGC50313.2020.9315807>.
172. Asadi S, Roshan S, Kattan MW (2021) Random forest swarm optimization-based for heart diseases diagnosis. *J Biomed Inform* 115:103690
173. Zhou M, Lin F, Hu Q, Tang Z, Jin C (2020) AI-enabled diagnosis of spontaneous rupture of ovarian endometriomas: A PSO enhanced random forest approach. *IEEE Access* 8:132253–132264. <https://doi.org/10.1109/ACCESS.2020.3008473>
174. Moldovan D (2020) Cervical cancer diagnosis using a chicken swarm optimization based machine learning method. In: 2020 International Conference on E-Health and Bioengineering (EHB), pp 1–4. IEEE
175. Dixit A, Mani A, Bansal R (2021) CoV2-detect-net : design of COVID-19 prediction model based on hybrid DE-PSO with SVM using chest X-ray images. *Inf Sci (Ny)* 571:676–692. <https://doi.org/10.1016/j.ins.2021.03.062>

176. D. Dong, Z. Ye, J. Su, S. Xie, Y. Cao, and R. Kochan, "A Malware Detection Method Based on Improved Fireworks Algorithm and Support Vector Machine," pp. 846–851, 2020.
177. X. Wu, "Research on the Influence of Crossover Probability and Mutation Probability in GA-SVM Model".
178. D. Moldovan, I. Anghel, T. Cioara, and I. Salomie, "Machine Learning in Manufacturing : Processes Classification Using Support Vector Machine and Horse Optimization Algorithm," 2020.
179. Moodi M, Ghazvini M, Moodi H (2021) Knowledge-Based Systems A hybrid intelligent approach to detect Android Botnet using Smart. Knowledge-Based Syst 222:106988. <https://doi.org/10.1016/j.knosys.2021.106988>
180. L. Zhao, M. Gai, and Y. Jia, "Classification of Multiple Power Quality Disturbances Based on PSO-SVM of Hybrid Kernel Function," vol. 10, no. 1, pp. 138–146, 2019.
181. E. Science, "Short-Term Wind Speed Prediction of Wind Farms Based on Particle Swarm Optimization Support Vector Machine Short-Term Wind Speed Prediction of Wind Farms Based on Particle Swarm Optimization Support Vector Machine," 2021, <https://doi.org/10.1088/1755-1315/804/3/032061>.
182. Alhija MA, Al-Baik O, Hussein A, Abdeljaber H (2024) Optimizing blockchain for healthcare IoT: a practical guide to navigating scalability, privacy, and efficiency trade-offs. Indones J Electr Eng Comput Sci 35(3):1773–1785. <https://doi.org/10.11591/ijeecs.v35.i3.pp1773-1785>
183. N. Kanwal *et al.*, "Equipping Computational Pathology Systems with Artifact Processing Pipelines: A Showcase for Computation and Performance Trade-offs," *ArXiv Prepr:2403.07743*, vol. 1, no. 1, 2024, <https://doi.org/10.1186/s12911-024-02676-z>.
184. A. A. Nelay and M. Turgeon, "A comprehensive study of auto-encoders for anomaly detection: Efficiency and trade-offs," *Mach. Learn. with Appl.*, vol. 17, no. May, p. 100572, 2024, <https://doi.org/10.1016/j.mlwa.2024.100572>.
185. P. Santosh Kumar Patra and B. Tripathy, "Hybrid optimal feature selection-based iterative deep convolution learning for COVID-19 classification system," *Comput. Biol. Med.*, vol. 181, no. August, p. 109031, 2024, <https://doi.org/10.1016/j.combiomed.2024.109031>.
186. O. Attallah, "Skin-CAD : Explainable deep learning classification of skin cancer from dermoscopic images by feature selection of dual high-level CNNs features and transfer learning," *Comput. Biol. Med.*, vol. 178, no. June, p. 108798, 2024, <https://doi.org/10.1016/j.combiomed.2024.108798>.
187. A. Juliette and A. R. Murugan, "Diagnosis of heart disease using oversampling methods and decision tree classifier in cardiology," *Res. Biomed. Eng.*, pp. 99–113, 2023, <https://doi.org/10.1007/s42600-022-00253-9>.
188. Sabharwal R, Miah SJ, Wamba SF (2022) Extending artificial intelligence research in the clinical domain : a theoretical perspective. Ann Oper Res. <https://doi.org/10.1007/s10479-022-05035-1>
189. Alqaissi EY, Alotaibi FS, Ramzan MS (2022) Modern machine-learning predictive models for diagnosing infectious diseases. Comput Math Methods Med 2022(1):2321
190. P. Jiang, S. Sinha, K. Aldape, S. Hannehalli, C. Sahinalp, and E. Rupp, "Big data in basic and translational cancer research", <https://doi.org/10.1038/s41568-022-00502-0>.
191. S. Marino, J. Xu, Y. Zhao, N. Zhou, Y. Zhou, and I. D. Dinov, "Controlled feature selection and compressive big data analytics : Applications to biomedical and health studies," pp. 1–21, 2018.
192. Q. Xu *et al.*, "Interpretability of Clinical Decision Support Systems Based on Artificial Intelligence from Technological and Medical Perspective : A Systematic Review," vol. 2023, 2023.
193. Yoon CH, Torrance R, Scheinerman N (2022) Machine learning in medicine: should the pursuit of enhanced interpretability be abandoned? J Med Ethics 48(9):581–585
194. Oyelade ON, Ezugwu AE (2022) Immunity - based Ebola optimization search algorithm for minimization of feature extraction with reduction in digital mammography using CNN models. Nature Publishing Group UK. <https://doi.org/10.1038/s41598-022-22933-3>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.